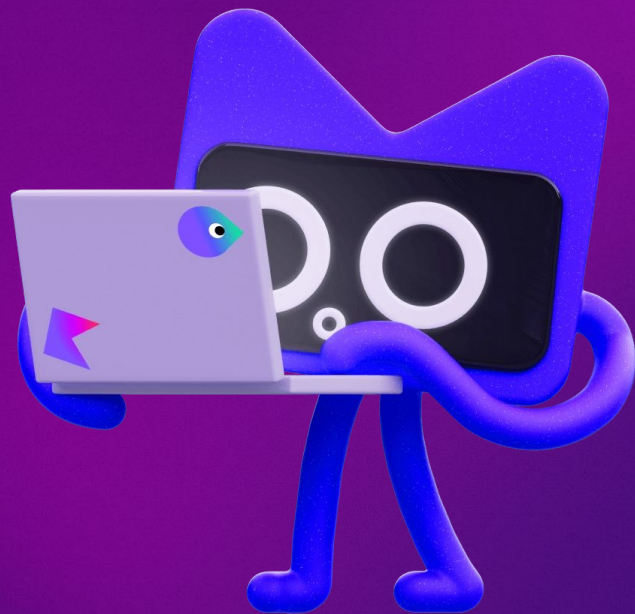




Techniques for seamless code sharing with Kotlin Multiplatform



About your instructor - Pamela



Kotlin Developer Advocate at **JetBrains**

Living in **Cape Town**, South Africa

About your mentor - Kostya



Software Developer in Compose Multiplatform at **JetBrains**

Living in **Leiden**, The Netherlands

Approximate Agenda (in GMT+1 timezone)

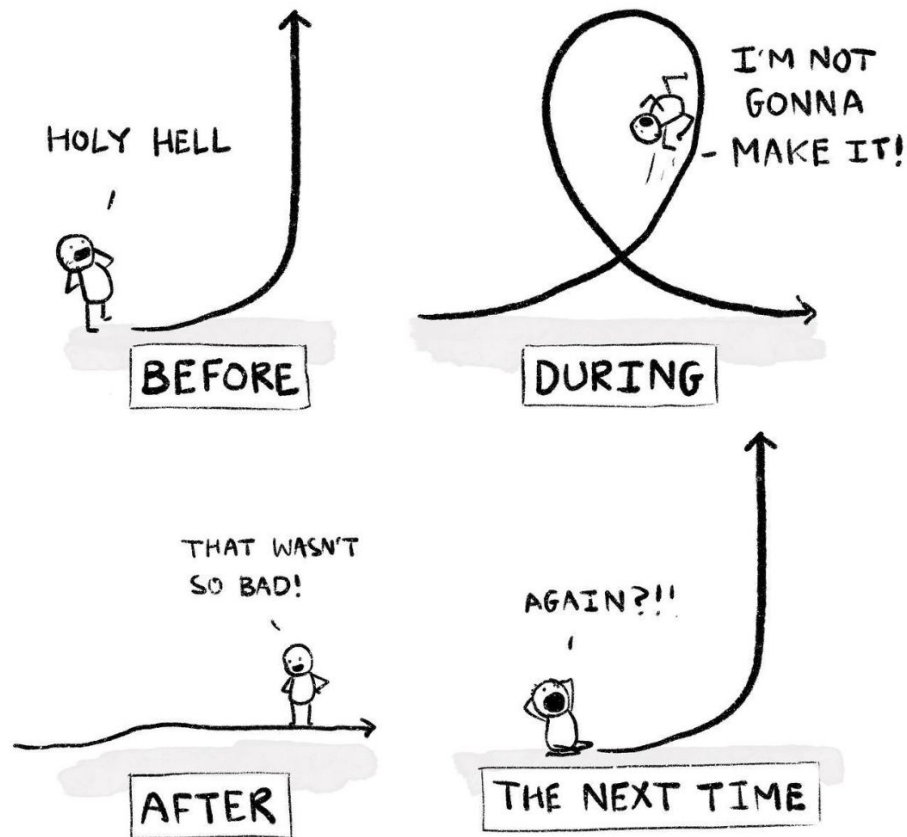
- Before we start: Setup verification
- 11:00 Welcome
- 11:01 Knowledge check and review
- 11:30 Content
- 13:00 Lunch
- 14:00 Content
- 15:30 Coffee break
- 16:00 Content
- 17:30 Q&A with Kostya
- 18:30 Discussion time, photo, goodbye
- 19:00 End of workshop

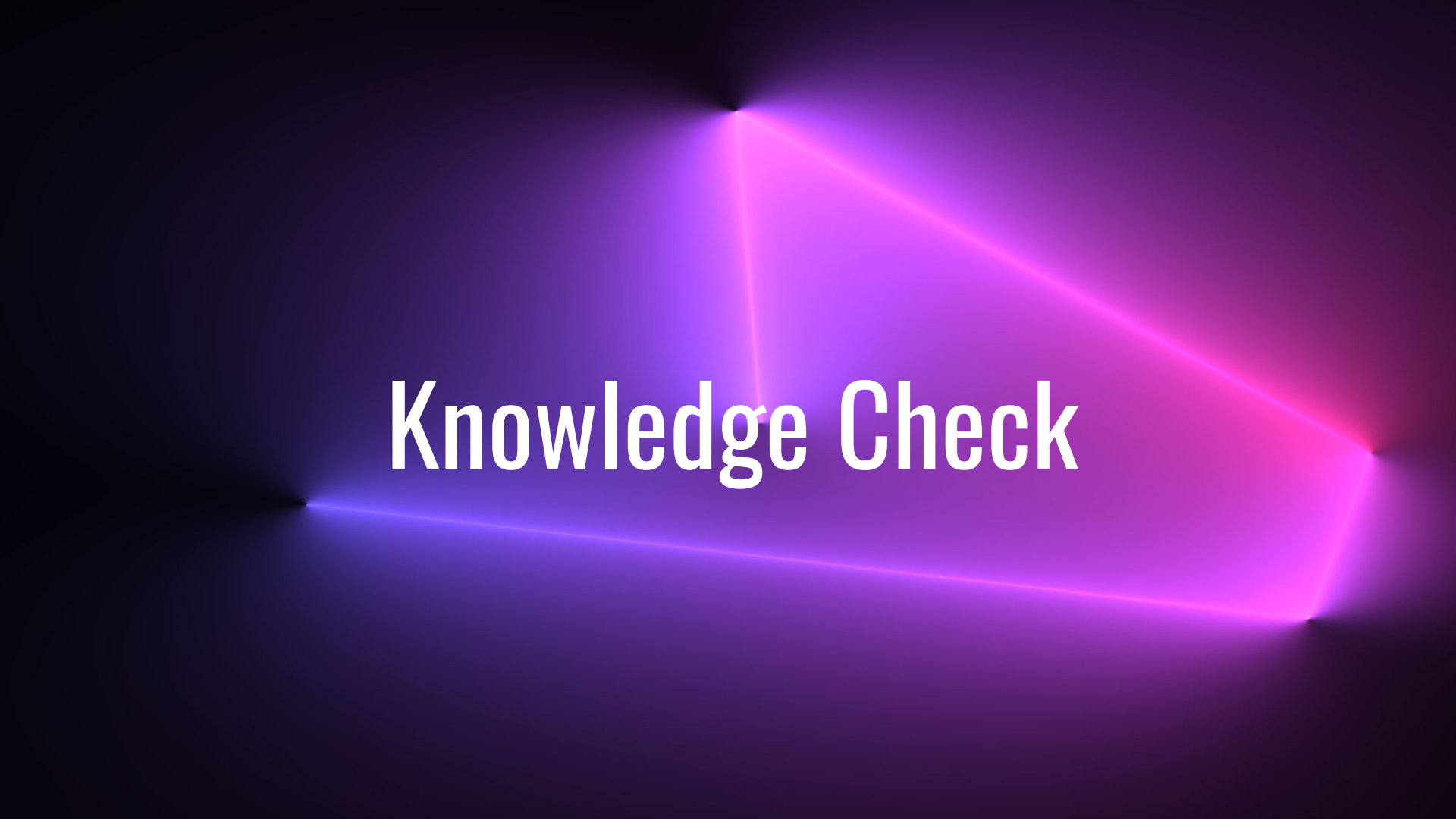
Tricky Questions for Kostya

Join at
slido.com
#4001 243



HOW LEARNING CURVES FEEL...



A glowing pink and purple 3D wireframe triangle is centered on a dark background. The triangle's edges are composed of bright, neon-like lines that create a strong sense of depth and perspective. The colors transition from a deep magenta at the top to a vibrant pink at the bottom, with a bright white highlight at the top vertex. The overall effect is futuristic and high-tech.

Knowledge Check

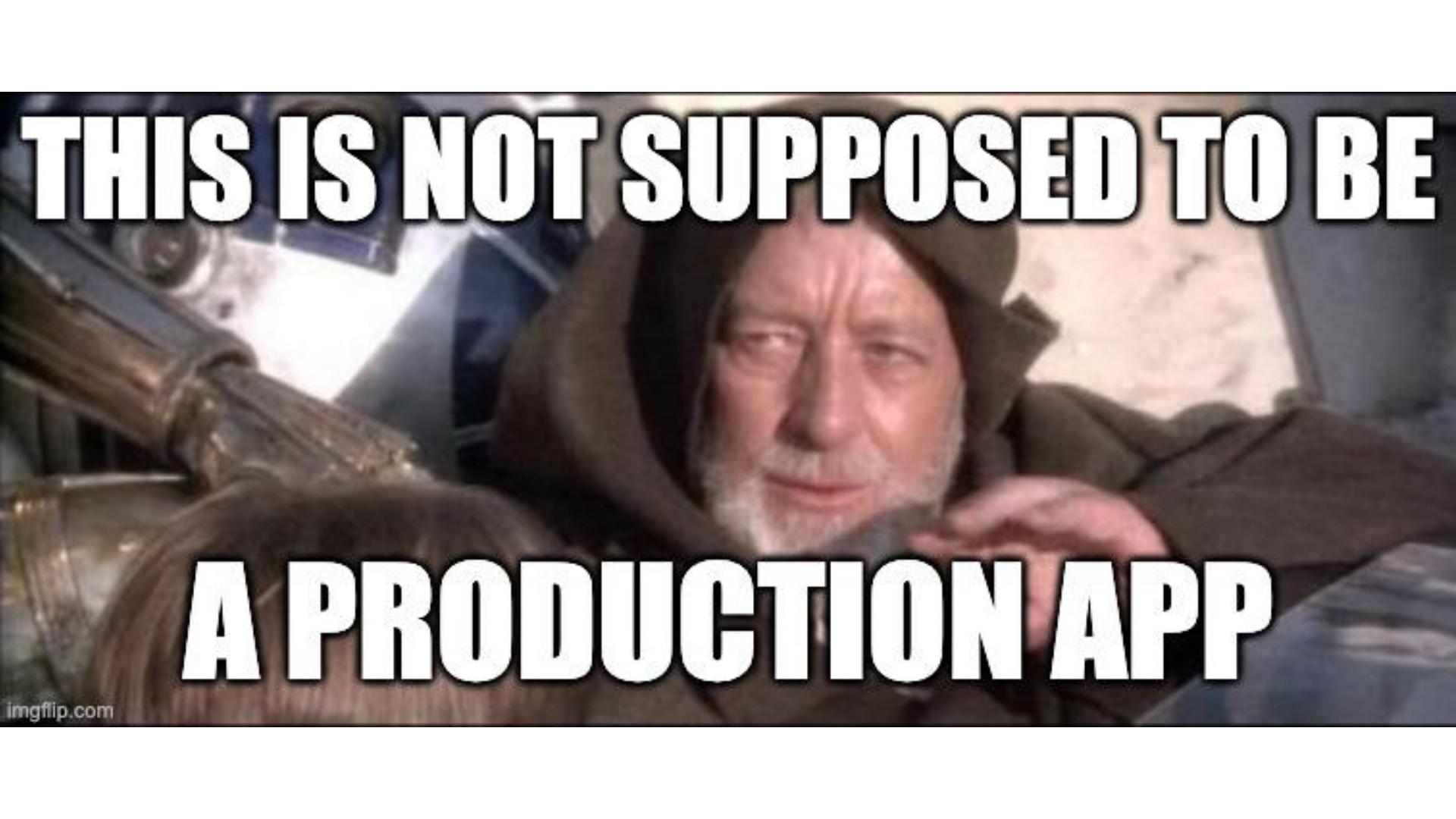
A glowing pink and purple 3D wireframe triangle is centered in the background. The triangle's edges are composed of bright, slightly blurred lines that create a sense of depth and light. The colors transition from a deep magenta at the top to a lighter pink at the bottom. The background is a dark, gradient purple, and the overall effect is ethereal and futuristic.

A Gentle(r) Introduction

What we'll be looking at:

- Introducing the example project.
- Explaining project structure and architecture.
- Highlighting a few key libraries used in the project.
- Feel free to explore the code if there is free time.

Introducing the example project



THIS IS NOT SUPPOSED TO BE

A PRODUCTION APP

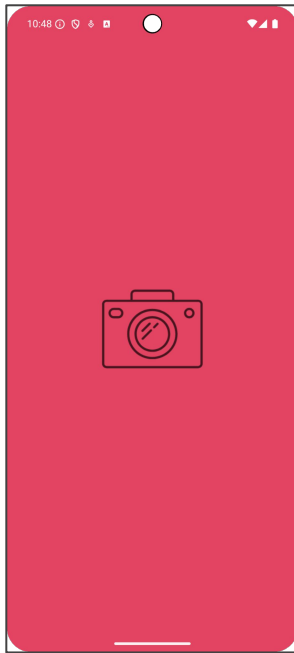
Camera app

- KMP/CMP app.
- Runs on Android / iOS / Desktop(JVM).
- Functionality:
 1. Allows you to get an image:
 - a. (Android/iOS Device) Take a photo using your native camera, OR
 - b. (iOS Sim/Desktop) Pick an image with a file picker.
 2. Apply some image filters to the image file.
 3. Create a local notification with the image file.

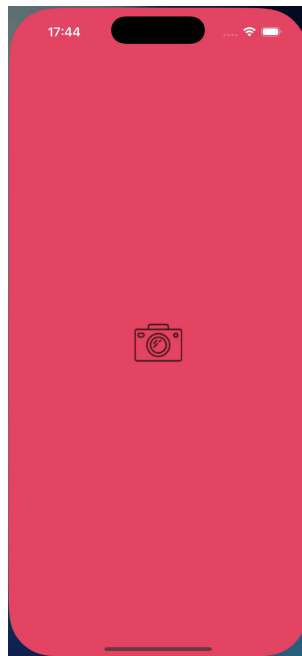
Step -1: Show the splash screen

— — —

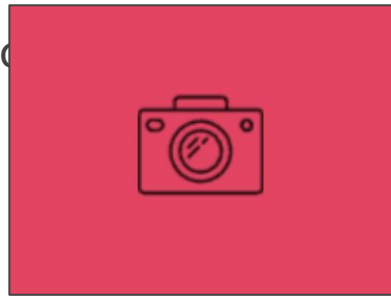
- Done natively.
- No animations - simple image with a background.



Android



iOS

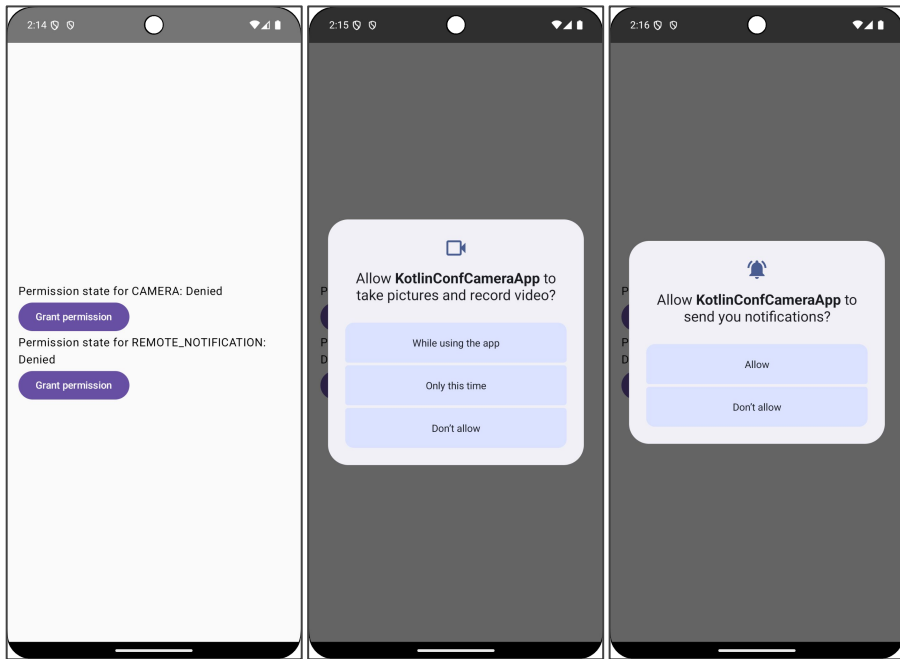


Desktop

Step 0: Permissions screen

— — —

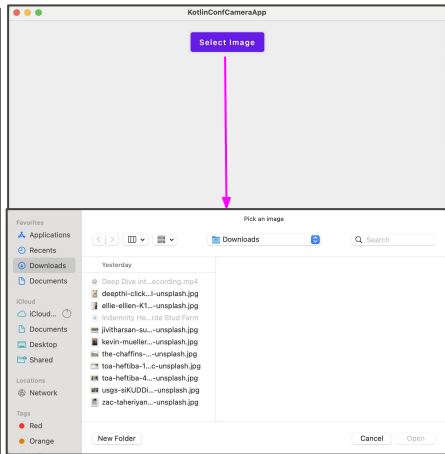
- Certain operations on Android/iOS need permissions.
- For example: camera, notifications, gallery.
- Android/iOS with CMP and library.
- Desktop handles permissions in another way.



Step 1: Camera screen

— — —

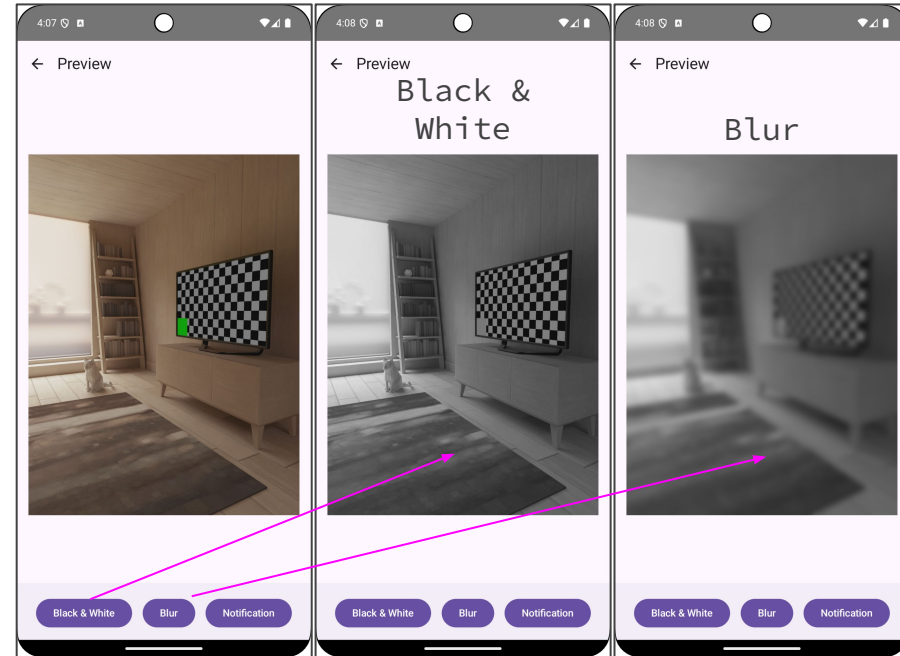
- Native cameras on Android/iOS device – live preview on back-facing camera.
- “Select image” button that opens file picker in user directory on Desktop/iOS sim.



Step 3: Pictures screen

— — —

- Preview of image taken/selected in previous screen.
- Two filters: black & white, blur.
- Done natively.



Step 3: Notifications popup

— — —

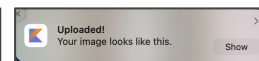
- Local notifications done natively using a library for Android/iOS/Desktop.



Android



iOS

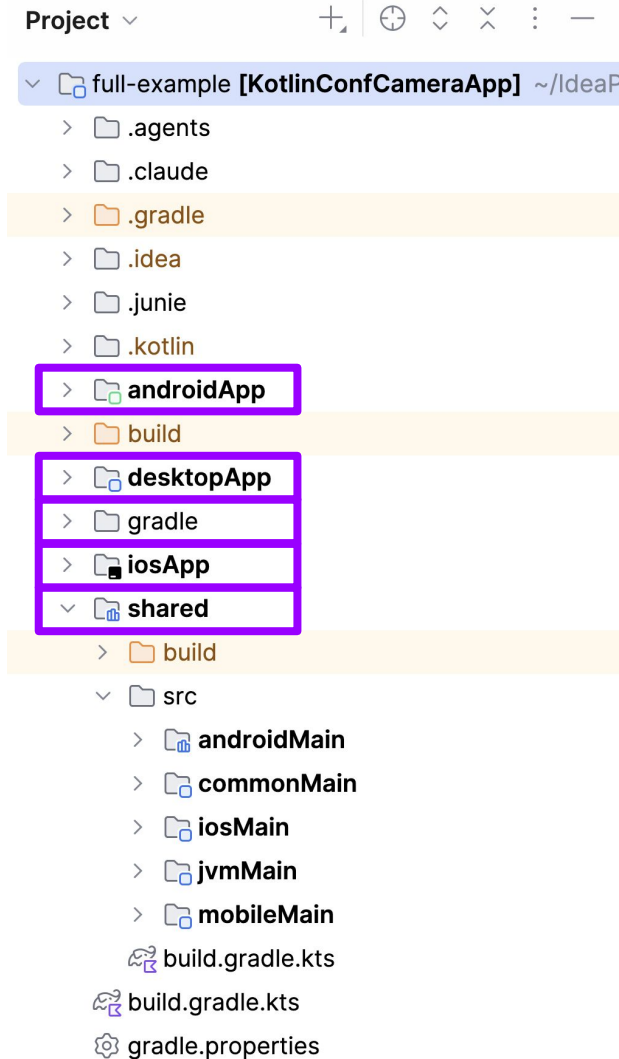


Desktop

Explaining project structure and architecture

Project structure [top-level]

- New AGP 9 structure.
 - androidApp / desktopApp / iosApp for the actual apps.
 - All apps use shared UI and logic, common code goes in **shared** module.
- gradle - directory with Gradle wrapper, version catalog etc.

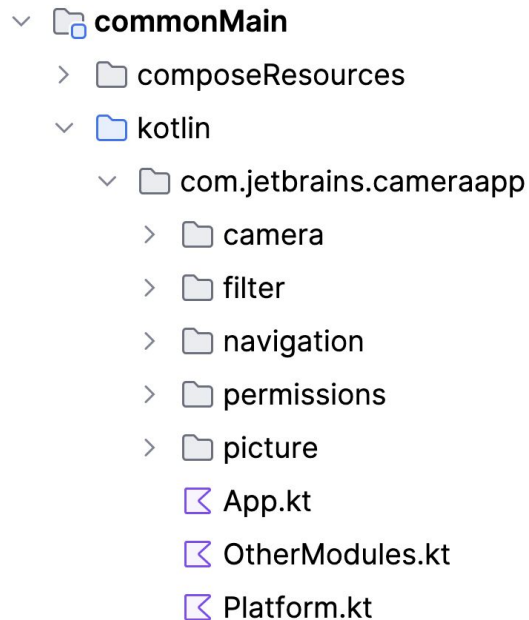


Project structure

[commonMain]

— — —

- Shared code and resources.
- We tried to put as much code here as possible.
- Packaged-by-feature rather than packaged-by-layer or type.

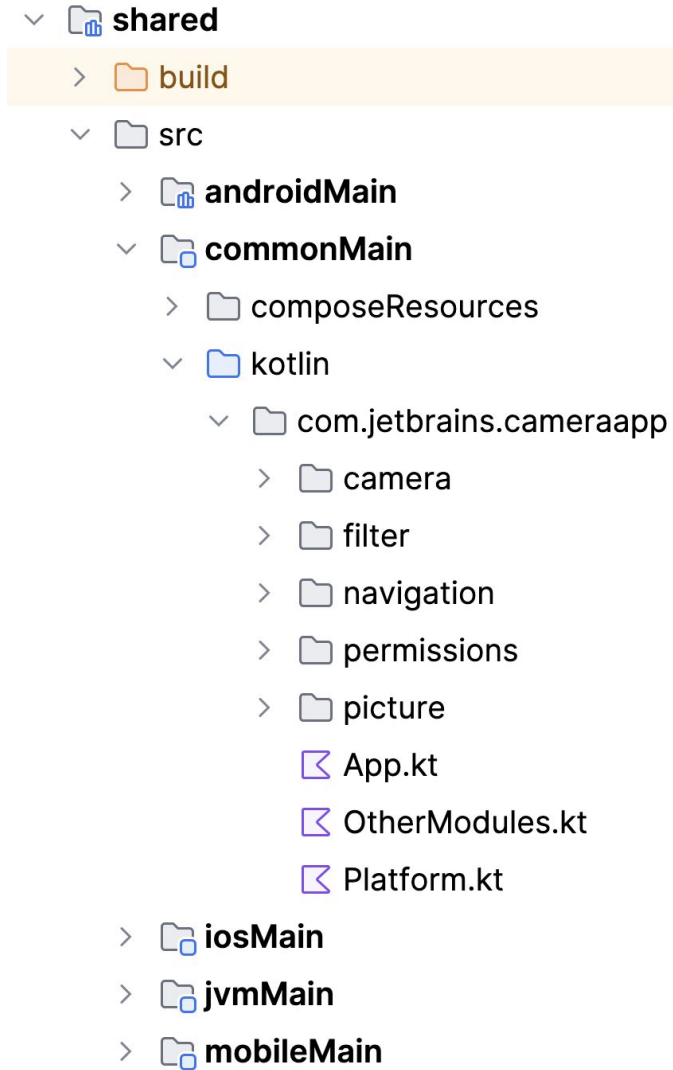


Project structure

[other source sets]

- androidMain / jvmMain / iosMain
– platform-specific code*.
- mobileMain – intermediate source set for Android/iOS-specific code* (related to permissions). Why?
 - Our library doesn't support Desktop.
 - Desktop handles permissions differently.

* Think actuals etc.



Highlighting a few key libraries used in the project

Interesting libraries used in the project [JetBrains/Google]

- Nav3 + Kotlinx-serialization - implementing navigation - [docs](#).
 - Migrated from Nav2 with Compose - also works for our scenario, but nice to know about.
 - Part of migration was with Gemini skill, but not good with MP scenario nuances.
 - User-defined and managed back stack.
 - More flexibility in implementing nav components and behaviour.
 - Works better with adaptive layouts.
- ViewModel for Compose: [docs](#).
 - Alternatives: [KMP-ObservableViewModel](#) | [Voyager](#) (also has VM)

Interesting libraries used in the project [MP Community]

- [Coil](#) – fancy async image loader
 - Used Multiplatform version (v3+).
 - Disabled disk/memory cache as we're working with latest version on disk (after filters).
 - Alternatives: [Sketch](#).
- [Moko Permissions](#) – permissions for Android/iOS (need to ask before using)
 - You need to manually add text to iOS for the dialog.
 - “Privacy – Camera Usage Description” in the plist file.

Interesting libraries used in the project [MP Community]

- [Koin](#) - simple to understand and setup in a small project.

Good to know:

[Metro](#) by Zac Sweers

- Compile-time dependency injection.
- Good [docs](#) / [sample](#) support.
- Metro [talk](#) at Droidcon London/KotlinConf to get started.

Community debate - DI vs SL



Zac Sweers (Creator of Metro)

8 min read · February 20, 2026

Re: Dependency Injection vs. Service Locators

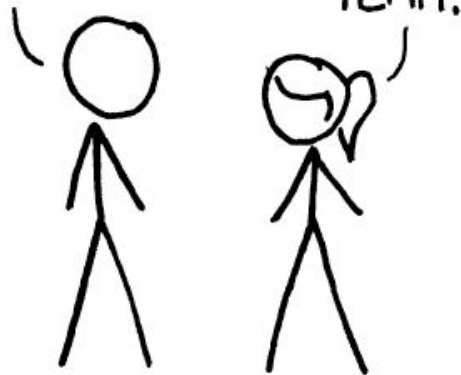
This is a port of a write-up I did in the Kotlin Lang slack [here](#) in response to the question "Dagger vs. Hilt vs. Koin vs. Metro vs. <what comes next>. Serious question: Are there any compelling reasons to switch from Koin to Metro in a Compose Kotlin multiplatform project?"

HOW STANDARDS PROLIFERATE:

(SEE: A/C CHARGERS, CHARACTER ENCODINGS, INSTANT MESSAGING, ETC.)

SITUATION:
THERE ARE
14 COMPETING
STANDARDS.

14?! RIDICULOUS!
WE NEED TO DEVELOP
ONE UNIVERSAL STANDARD
THAT COVERS EVERYONE'S
USE CASES.



YEAH!

SOON:

SITUATION:
THERE ARE
15 COMPETING
STANDARDS.

Interesting libraries used in the project [MP Community]

- KMPNotifier
 - Local / push notifications. For our purposes - just local.
 - You do need to do some Firebase setup.
 - iOS: plist download, Firebase SPM packages addition.
 - Android: google-services.json.
 - Notification images are inconsistent.
 - Desktop: run with **runDistributable**.
 - Alternatives: [Alarmee](#) (good docs)| do-it-yourself.

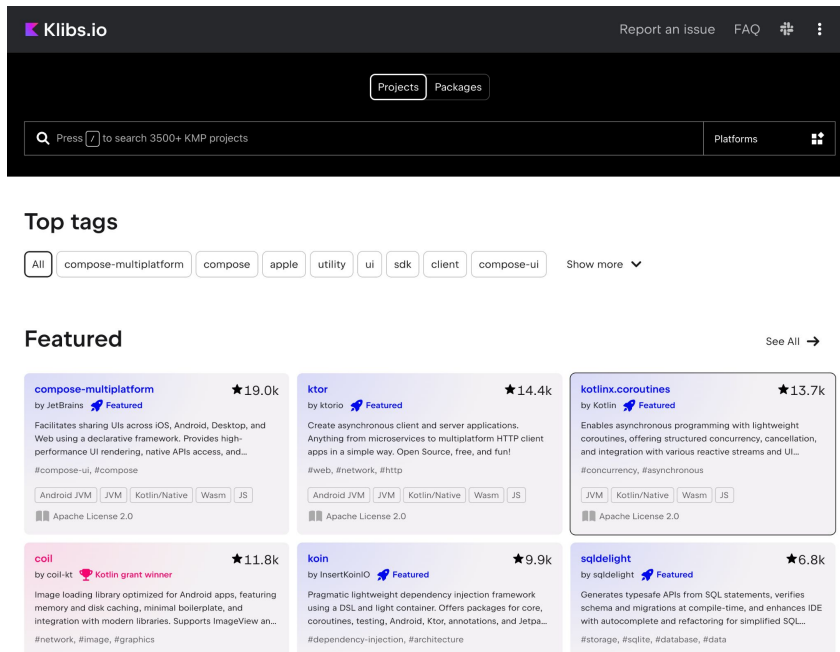
Interesting libraries used in the project [Desktop-specific]

- [FileKit](#) - Multiplatform file operations, pickers, dialogs.
 - We used the picker for Desktop/iOS Sim, but easy to extend to other platforms.
 - Better native look than Swing components.

How to get libraries [klibs.io]

— — —

- Official from JetBrains.
- GitHub projects, automatically added.
- Look out for:
 - Kotlin Foundation grant winners – impactful / supported financially.
 - Featured projects – curated collections by JetBrains team.



A glowing pink and purple 3D wireframe triangle is centered on a dark background. The triangle's edges are composed of bright, slightly blurred lines that create a sense of depth and light. The colors transition from a deep magenta at the top vertex to a lighter, more vibrant pink towards the bottom right vertex. The overall effect is ethereal and futuristic.

Native Integrations

What we'll be looking at:

- Adding splash screens.
- Working with files.
- Sending a local notification.

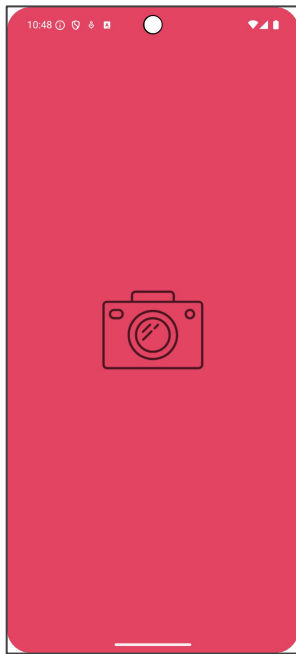
These seem like quite arbitrary tasks, why **them**?

Adding Splash Screens

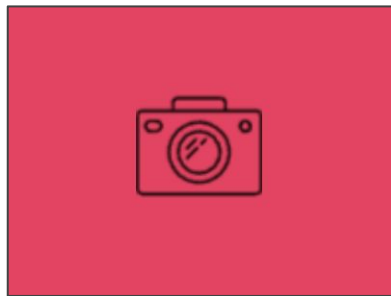
Splash screens

— — —

- Splash screens are there for a quick flash of branding, **not** for a lot of loading data.
- Troubleshooting:
 - Android / iOS: You may need to kill the app (swipe up) / even reinstall.
 - Desktop: make sure you are running with **runDistributable**.



Android



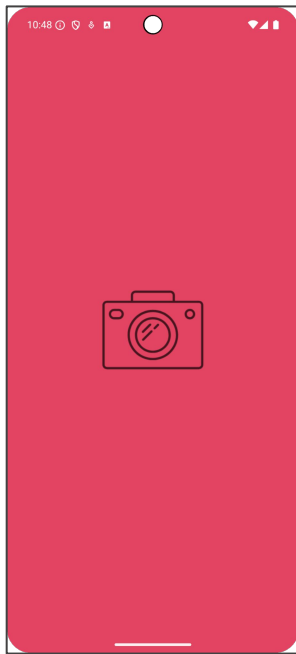
Desktop

Splash screens

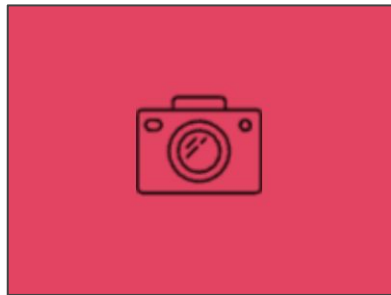
— — —

- Splash screens are there for a quick flash of branding, **not** for a lot of loading data.
- Troubleshooting:
 - Android / iOS: You may need to kill the app (swipe up) / even reinstall.
 - Desktop: make sure you are running with **runDistributable**. So:

```
./gradlew desktopApp:runDistributable
```



Android



Desktop

Steps for Android [static]

1. Add the splash screen library
2. Import the .svg file as a vector asset
3. Make the vector drawable fit according to the Google guidelines
4. Create a splash screen theme
5. Apply the splash screen theme to the application and first Activity
6. Install the splash screen
7. Run & profit

Steps for iOS [static]

1. Open project in Xcode
2. Add the image to your Xcode project
3. Create the Launch Screen in Xcode properties
4. Run & profit

▼ Launch Screen	◇ Dictionary	◇ (3 items)
Image Name	◇ String	◇ camera
Background color	◇ String	◇ SplashColor
Image respects safe area insets	◇ Boolean	◇ YES

Steps for Desktop [static]

1. Add the png image to assets/common in desktopApp
2. Update the build.gradle.kts file for compose.desktop:
 - a. Point to appResourcesRootDir to the assets directory
 - b. Add a jvmArg for the splash screen
3. `./gradlew desktopApp:runDistributable` and profit

Practical 1: Splash Screens

[40 mins]

Notifications

Local Notifications

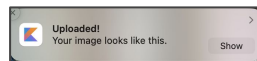
- Notifications done using a library for Android/iOS/Desktop.
- Troubleshooting:
 - Desktop: make sure you are running with **runDistributable**.



Android



iOS



Desktop

Demo time

KMPNotifier by Mirzamehdi Karimov

— — —

- Simple & easy-to-use.
- Supports local and push notifications (we'll only look at local ones).
- Local: Android, iOS, Desktop and Web (JS and Wasm).
- Push: Android, iOS.



KMPNotifier

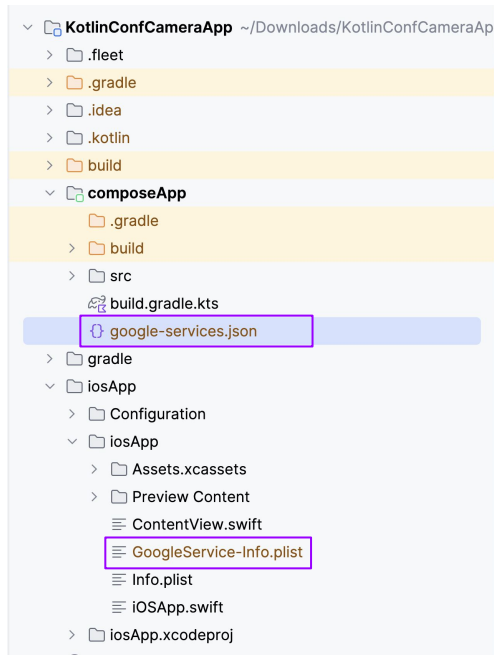
Steps to use KMPNotifier

0. Setup project in [Firebase Console](#).
 - Download and add google-services.json file for Android.
 - Download and add GoogleService-Info.plist for iOS.
1. Initialize the library.
2. Send the notification.

Setup project in Firebase Console

— — —

- Go to [Firebase Console](#).
- Create the project as usual in Firebase.
- Download and add google-services.json file for Android.
- Download and add GoogleService-Info.plist for iOS.



Initialize the library - Android

- “First” thing in application onCreate:

```
NotifierManager.initialize(  
    configuration = NotificationPlatformConfiguration.Android(  
        notificationIconResId = R.drawable.camera_splash,  
        showPushNotification = false,  
    )  
)
```

) Branding icon: top bar and left of the notification body.

Initialize the library - Desktop

- “First” thing in **application** lambda:

```
NotifierManager.initialize(  
    NotificationPlatformConfiguration.Desktop(  
        showPushNotification = false,  
        notificationIconPath = composeDesktopResourcesPath() + File.separator + "splash.png"  
    )  
)
```

System.getProperty("compose.application.resources.dir")



[Docs](#)

Initialize the library - iOS - Create a delegate class

- In `iOSApp.swift`:

```
class AppDelegate: NSObject, UIApplicationDelegate {
    func application(_ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey : Any]? = nil) → Bool
    {

        FirebaseApp.configure() //important

        NotifierManager.shared.initialize(configuration: NotificationPlatformConfigurationIos(
            showPushNotification: false,
            askNotificationPermissionOnStart: false,
            notificationSoundName: nil
        ))
        return true
    }

    func application(_ application: UIApplication,
        didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
        Messaging.messaging().apnsToken = deviceToken
    }
}
```

UIApplicationDelegate is a protocol in iOS development that defines a set of methods for managing and responding to the **lifecycle events** of an iOS application.

It acts as the central hub for handling events such as app launch, termination, entering background or foreground, receiving push notifications, and more.

Initialize the library - iOS - Setup delegate in app

- In iOSApp.swift:

```
@main
struct iOSApp: App {
```

Swift property wrapper
(similar to property
delegate in Kotlin)



```
    @UIApplicationDelegateAdaptor(AppDelegate.self) var delegate
```

```
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Send the notification

- In commonMain:

```
val notifier = NotifierManager.getLocalNotifier()
notifier.notify {
    id= Random.nextInt(0, Int.MAX_VALUE)
    title = "Uploaded!"
    body = "Your image looks like this."
    image = NotificationImage.File(imagePath)
}
```



Other option is Url

Nice-looking alternative: Alarmee

— — —

- **Docs look more elaborate.**
- Supports local and push notifications (Firebase Messaging for Android / Apple Push Notification for iOS)
- Local: Android, iOS, Desktop and Web.
- Push: Android, iOS.



ALARMEE

Question: When should we choose a multiplatform library and when should we go native?

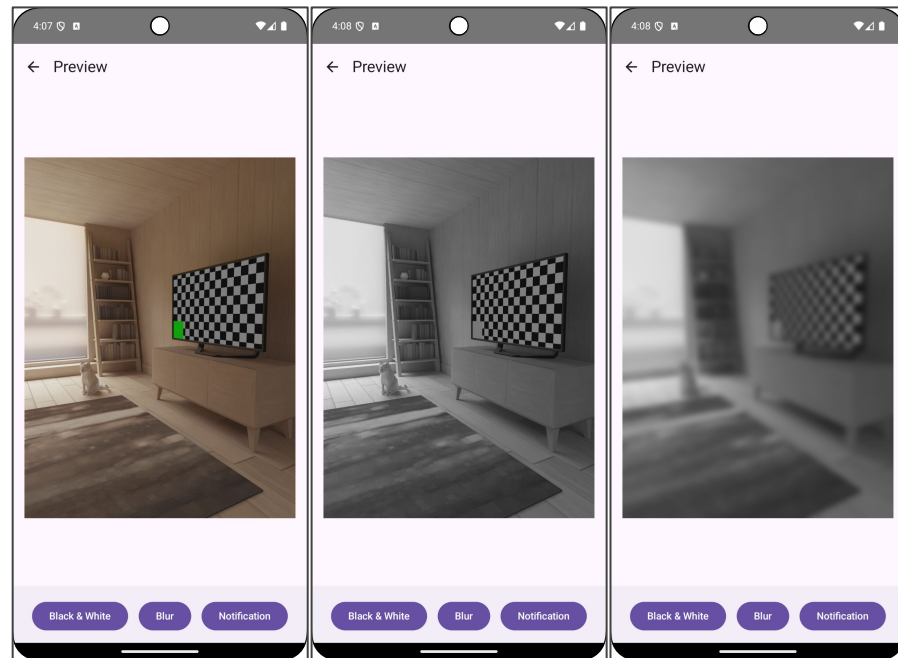
Managing Files

Managing Files - Read/Filter/Write

— — —

Applying filters:

1. Check exists
2. Open, read
3. Filter based on filter type and platform
4. Save file in same spot
5. AsyncImage is updated - no caching!



Demo time

General how to decide

- What to use? Depends on your tasks!
 - If you need a raw file management (read/write buffers), then [kotlinx-io](#).
 - If you need a platform Filepicker UI to select a directory or file, then it's easier with [FileKit](#).
 - FileKit can also be used *without* the picker, and has a nice abstraction on top of the filesystem.
 - If you need something else, then the native.

Our thought process

- We wanted to do **image filtering on files**, but having some platforms support **file picker** would be nice.
- What options do we have?
 - Kotlinx-io feels too low-level for image filtering.
 - Using an image filter library like Peekaboo (but Android/iOS only).
 - FileKit + Native filters for each platform, all the filters we want is already supported per platform.

Our thought process

- We wanted to do image filtering on files, but having some platforms support file picker would be nice.
- What options do we have?
 - Kotlinx-io feels too low-level for image filtering.
 - Using an image filter library like Peekaboo (but Android/iOS only).
 - **FileKit + Native filters for each platform, all the filters we want is already supported per platform.**

Android Solution

— — —

1. Validation - Android
2. Load bitmap - Android
3. Create (copy) bitmap - Android
4. Create canvas - Android
5. Apply filter - unique
6. Draw bitmap - unique
7. Save bitmap - Android

For example - B&W filter:

android.graphics

```
val paint = Paint().apply {  
    // Create a color matrix that  
    // converts to grayscale  
    val colorMatrix = ColorMatrix().apply {  
        setSaturation(0f)  
    }  
    colorFilter =  
        ColorMatrixColorFilter(colorMatrix)  
}
```

Gaussian blur: RenderScript

Desktop Solution

— — —

1. Validation - Desktop
2. Load image - Desktop
3. Apply filter - unique
4. Save image - Desktop

For example - B&W filter:
`java.awt.image`

```
val blackAndWhiteImage =  
    BufferedImage(width, height,  
        BufferedImage.TYPE_BYTE_GRAY)
```

Gaussian blur: also `java.awt.image`
with own Gaussian kernel.

iOS Solution

— — —

1. File validation - iOS
2. Rest - varies a bit but UIKit & CoreGraphics

B&W: Color space for greyscale

Gaussian blur: Filter

Additional info about FileKit

- Key abstraction: **PlatformFile** - a unified representation of files across all platforms.
- Abstracts away platform-specific file implementations, provides a consistent API for working with files in KMP.
- Reading / writing files.
- File utilities: platform-specific directory behaviour - files, caches, dbs.
- Image / video utilities: compression, saving to gallery, thumbnails.

The background features a dark, gradient purple field. Overlaid on this are several glowing, translucent geometric shapes. A prominent shape is a large, irregular polygon with vertices that appear to be light sources, emitting a bright pinkish-purple glow. Other smaller, similar shapes are visible, creating a sense of depth and modern, tech-oriented design.

Advanced Kotlin Multiplatform for iOS Targets

What we'll be looking at:

- Debugging with `xcode-kotlin`.
- Further debugging with LLDB.
- Understanding and improving Kotlin/Swift interoperability (including a look at Swift Export).

Debugging in Xcode

backstack

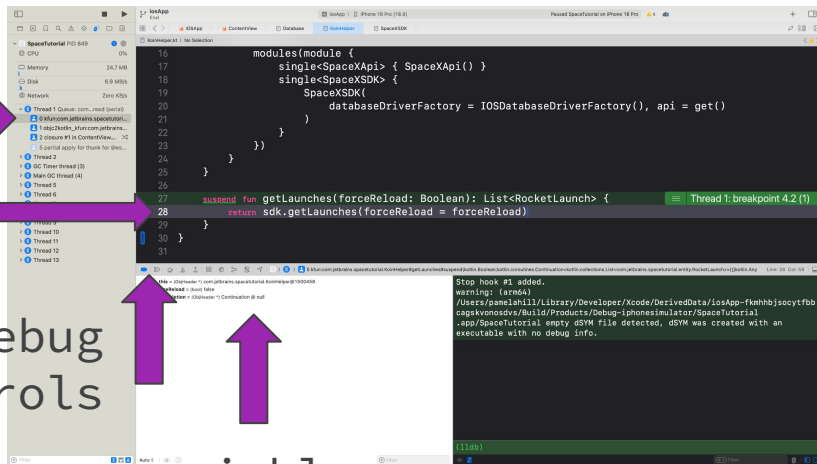
line numbers
(can create
breakpoints)

debug
controls

variables

this is
draggable

LLDB
console



Debugging with xcode-kotlin

Introduction to xcode-kotlin

- Open-source tool from Touchlab.
- Plugin to improve the Kotlin/Swift debugging experience.
- Supports some common debugging activities.
- Doesn't support state inside suspend functions so well.

Installation / Syncing (Terminal)

- Installation using Homebrew:

```
brew install xcode-kotlin  
xcode-kotlin install
```

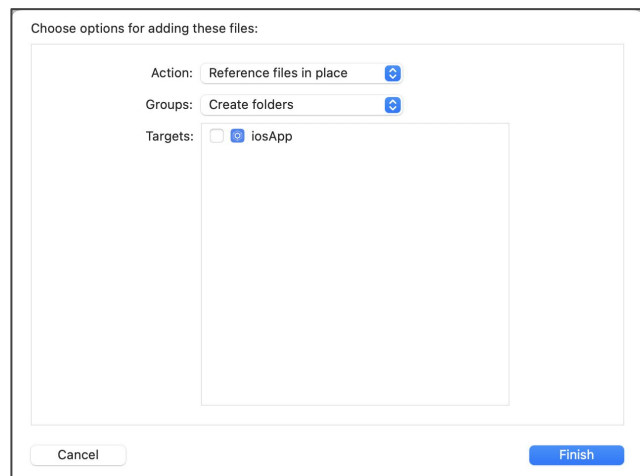
- Syncing with new Xcode versions:

```
xcode-kotlin sync
```

Demo time

Setup of sources in xcode-kotlin

1. Right-click -> New Group.
2. Right-click -> Add files to iOS App.
3. Select commonMain and iosMain.
4. Set Action to “Reference files in place”.
5. Set Groups to “Create folders”.
6. Make sure iosApp is **unchecked** in Targets.
7. Finish.



Extra resources

- <https://touchlab.co/xcodekotlin>
- <https://touchlab.co/xcode-kotlin-2-0>
- #touchlab-tools channel on Kotlinlang

Debugging with LLDB

Introduction to LLDB

- Powerful debugger environment for Apple platforms.
- Debug Swift / Objective-C / C / C++ code.
- Only on macOS platforms, from terminal / Xcode.

Simple breakpoint commands

- Breakpoint at line number: `b line_number`
- Breakpoint at function: `b function_name`
- Breakpoint in file at line: `b -f filename -l linenumber`
- Breakpoint list: `breakpoint list` (the first number is the id of the breakpoint)
- Disable breakpoint: `breakpoint disable id`
- Enable breakpoint: `breakpoint enable id`
- Delete breakpoint: `breakpoint delete id`

Simple debugging

- Help: `help`
- Continue (all threads): `c`
- Next step with into (source-level): `step`
- Step over (source-level): `n`
- Step out (source-level): `finish`
- Print variable or expression: `p variable|expression`
- Print expression with formatting: `po expression`
- Backtrace: `bt`

Connecting to a simulator, creating a breakpoint

```
xcrun simctl launch booted com.jetbrains.cameraapp.KotlinConfCameraApp
com.jetbrains.cameraapp.KotlinConfCameraApp: 21905
lldb
(lldb) process attach -p 21905
(lldb) b -f GaussianBlurFilter.ios.kt -l 11
(lldb) continue
```

Launch sim

Port number

Attach to process at port number

Filename

Line number

Run till breakpoint

Loading konan_lldb.py for pretty printing values

```
(lldb) command script import  
/Users/pamelahill/.konan/kotlin-native-prebuilt-macos-aarch64  
-2.3.21/tools/konan_lldb.py
```

Version
of
Kotlin
you used

Your
username

Your
architecture

But what is the best way? UI vs CLI

- It depends! BUT
- Learn to use LLDB because
 - UI doesn't support everything, for example:
 - Regex breakpoints
 - Expression evaluation for Kotlin/Native
 - If you want to shell script LLDB itself in the future

Practical 2: Debugging Kotlin/Swift [20 mins]

Other resources

- [Effective Debugging Kotlin Native in Xcode - Tadeas Kriz](#)
- [\(Kotlin docs\) Debugging Kotlin/Native](#)
- [Documentation from Apple](#)
- [Learn the lldb debugger basics in 11 minutes | 2021 \(Also works on M1 Apple Silicon\) - Mike Shah](#)

Understanding and Improving Kotlin/Swift Interoperability

What we'll be looking at:

- Short (and hopefully painless) history lesson about Kotlin/Swift interop - skippable.
- Discuss and demo Swift Export (current state).

What exactly is the problem?

- Decision made in ~2018 to be interoperable with Objective-C, not Swift.
- Why?
 - Can be used in all iOS projects.
 - Swift still on the road to maturity/adoption at that stage.

Kotlin is **not directly** interoperable with Swift,
but **indirectly** via an Objective-C bridge.

What this means for Swift developers

Some Kotlin features:

- Work exactly as expected
- Work with a workaround
- Work better with a community solution
- Don't work optimally right now
- Don't work



Kotlin/Swift Interopedia [kotlin.in/interopedia]

- Interoperability encyclopedia
- Explains how Kotlin features currently work from Swift
- Lots of code samples
- Gives workarounds, community solutions
- Includes playground app with runnable samples

Overview

Classes and functions	You can instantiate Kotlin classes and call Kotlin functions from Swift: <code>SimpleClass().simpleFunction()</code> .
Top-level functions	You can access a top-level function via the wrapper class: <code>TopLevelFunctionKt.topLevelFunction()</code> .
Types	Simple types and custom types can be passed as arguments and returned from function calls.
Collections	Kotlin and Swift have very similar kinds of collections and can be mapped between each other.
Exceptions	If you invoke a Kotlin function that throws an exception and doesn't declare it with <code>@Throws</code> , that crashes the app. Declared exceptions are converted to <code>NSError</code> and must be handled.
Public API	Public classes, functions, and properties are visible from Swift. Marking classes, functions, and properties internal will exclude them from the public API of the shared code, and they will not be visible in Swift.
Interop annotation - @ObjCName	Gives better Objective-C/Swift names to Kotlin constructs like classes, functions and so on, without actually renaming the Kotlin constructs. Experimental.
Interop annotations - @HiddenFromObj	Hides a Kotlin declaration from Objective-C/Swift. Experimental.
Interop annotations - @ShouldRefineInSwift	Helps to replace a Kotlin declaration with a wrapper written in Swift. Experimental.
KDoc comments	You can see certain KDoc comments at development time. In Xcode, use Option+Double left click to see the docs. Note that many KDocs features don't work in Xcode, like properties on constructors (<code>@property</code>) aren't visible. In Fleet, use the 'Show Documentation' action.

KMP-NativeCoroutines - Rick Clephas & team

- Addresses issues related to suspend functions and Flows.
- Requires annotations, wrapper functions.
- Can also be used with Combine & RxSwift.
- Very quick updates to latest Kotlin versions.

Contributors 4



rickclephas Rick Clephas



ankushg Ankush Gupta



veyndan Veyndan Stuart



litclimbing Lit Climbing

SKIE [Swift Kotlin Interface Enhancer] - TouchLab

- Broader solution:
 - Also suspend functions, Flows without wrapper functions.
 - Enums: translated to Swift enums, allows for exhaustive switches.
 - Sealed hierarchies: handles exhaustive switches.
 - Better handling of functions:
 - Global
 - Interface extensions
 - Overloaded
 - Default arguments (not recommended)
- Mostly, you only have to include the plugin.



Swift Export

Goal:

**To deliver a more pleasant experience
for iOS developers out-of-the-box**

Experimental Features (Kotlin 2.2.20)

Simple classes	Multiple modules	Type aliases	Exceptions
Properties	Nested classes	Sealed classes	Function overloads
Primitive types	Top-level functions	Lambdas with receivers	
Packages	Function params	Extensions	
Data classes	Inner classes	Enums	
Methods	Objects	Companion objects	
Collection types	Top-level properties	Interfaces	

Covered features (alpha - Kotlin 2.4)

Simple classes	Multiple modules	Type aliases	Exceptions
Properties	Nested classes	Sealed classes	Function overloads
Primitive types	Top-level functions	Lambdas with receivers	
Packages	Function params	Extensions	
Data classes	Inner classes	Enums	
Methods	Objects	Companion objects	
Collection types	Top-level properties	Interfaces	

Functional types

varargs

Context params

Inline functions

Exhaustive enums

Suspend functions

Value classes

Flows

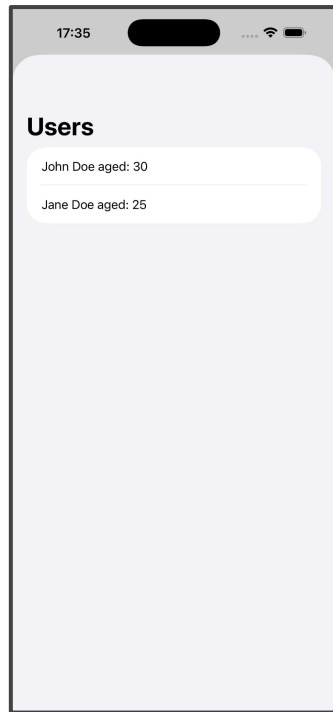
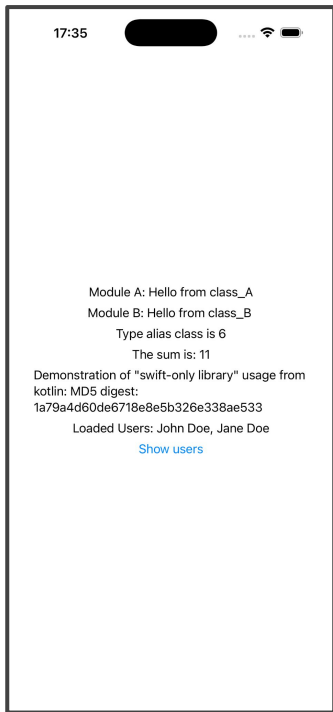
Graph key

Beta, future

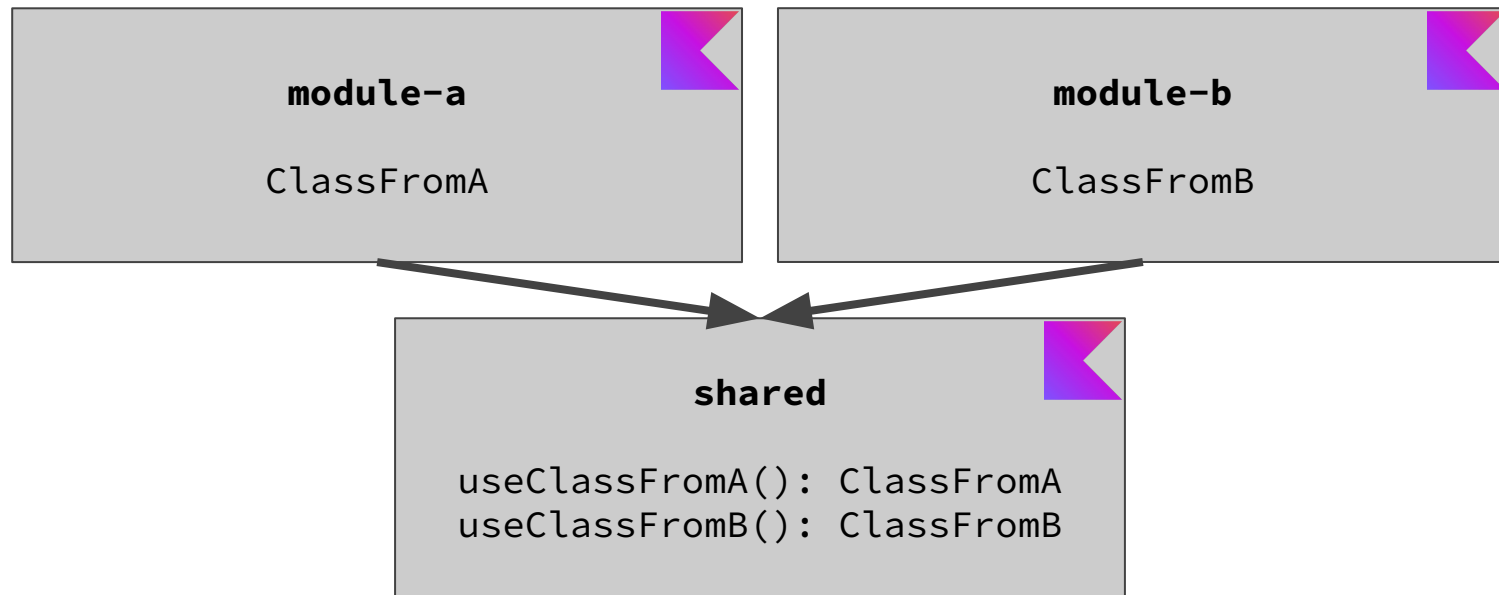
Alpha, this year: Kotlin 2.4

Last year: Kotlin 2.2.0

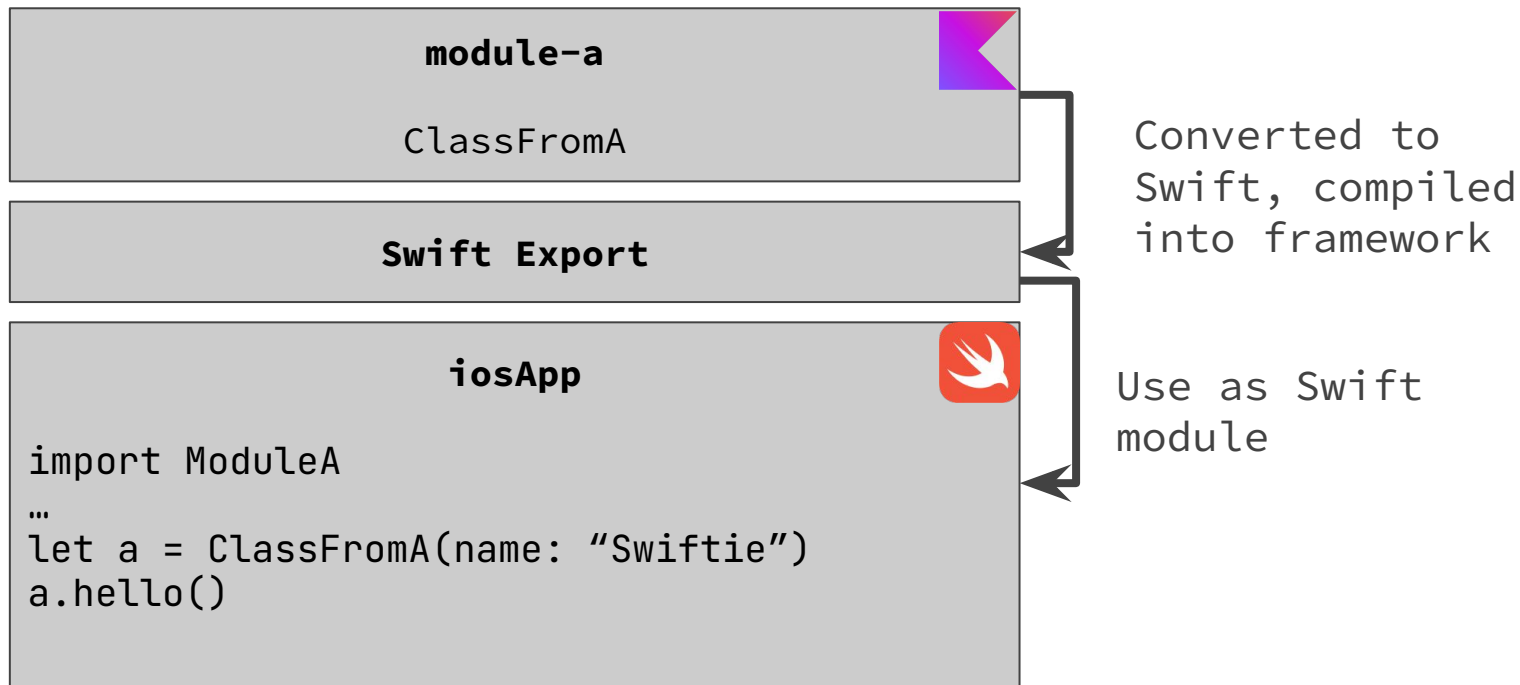
Swift Export sample app [kotlin.in/swift-export-sample]



General Structure of App



Modules & Packages

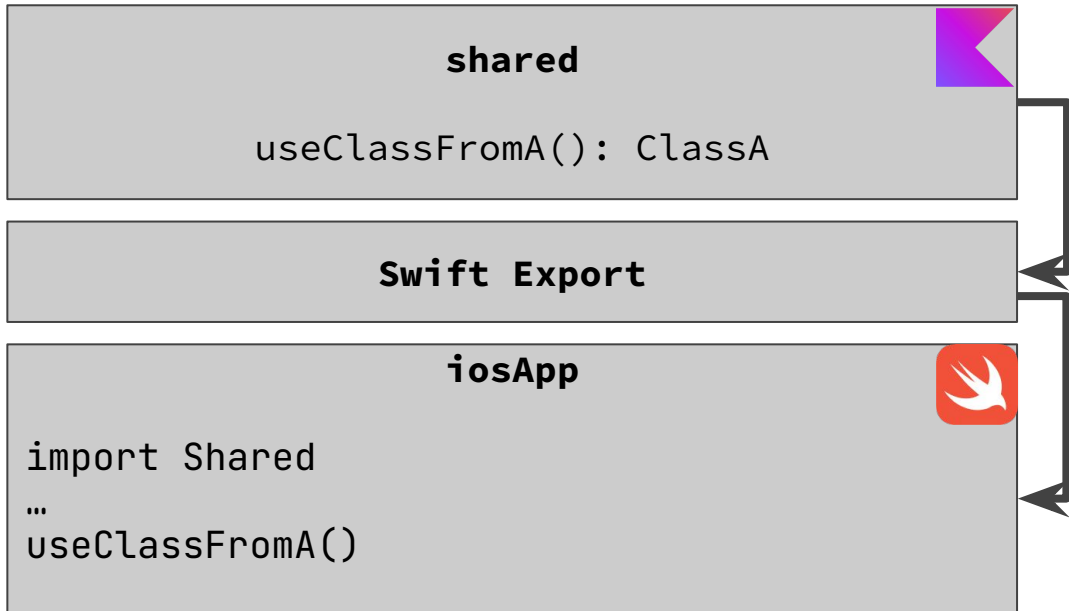


Top-level Function

Previously (example from Interopedia)

```
TopLevelFunctionKt.topLevelFunction()
```

Now with Swift Export



Function Overloads

Previously (example from Interopedia)

```
OverloadsKt.overloadFunction(param: true) // Bool  
OverloadsKt.overloadFunction(param_: 2.0) // Double  
OverloadsKt.overloadFunction(param__: 2.0) // Float  
OverloadsKt.overloadFunction(param___: 2) // Int32  
OverloadsKt.overloadFunction(param____: 4) // Int64  
OverloadsKt.overloadFunction(param_____: "123")
```

Now with Swift Export

shared

```
fun overloaded(x: String) {}  
fun overloaded(x: Int) {}  
fun overloaded(x: Double) {}
```



Swift Export

iosApp

```
import Shared  
...  
overloaded(x: "hello")  
overloaded(x: Int32(12))  
overloaded(x: 34.3)
```



Primitive Support

Previously (from Interopedia)

- `Byte` -> `Int8`
- `UByte` -> `UInt8`
- `Short` -> `Int16`
- `UShort` -> `UInt16`
- `Int` -> `Int32`
- `UInt` -> `UInt32`
- `Long` -> `Int64`
- `ULong` -> `UInt64`
- `Byte?` -> `KotlinByte?`
- `UByte?` -> `KotlinUByte?`
- `Short?` -> `KotlinShort?`
- `UShort?` -> `KotlinUShort?`
- `Int?` -> `KotlinInt?`
- `UInt?` -> `KotlinUInt?`
- `Long?` -> `KotlinLong?`
- `ULong?` -> `KotlinULong?`
- `Float?` -> `KotlinFloat?`
- `Double?` -> `KotlinDouble?`
- `Boolean?` -> `KotlinBoolean?`

Now with Swift Export

shared

```
fun callWithByte(b: Byte?) { println(b) }  
fun callWithInt(i: Int?) { println(i) }  
fun callWithLong(l: Long?) { println(l) }
```

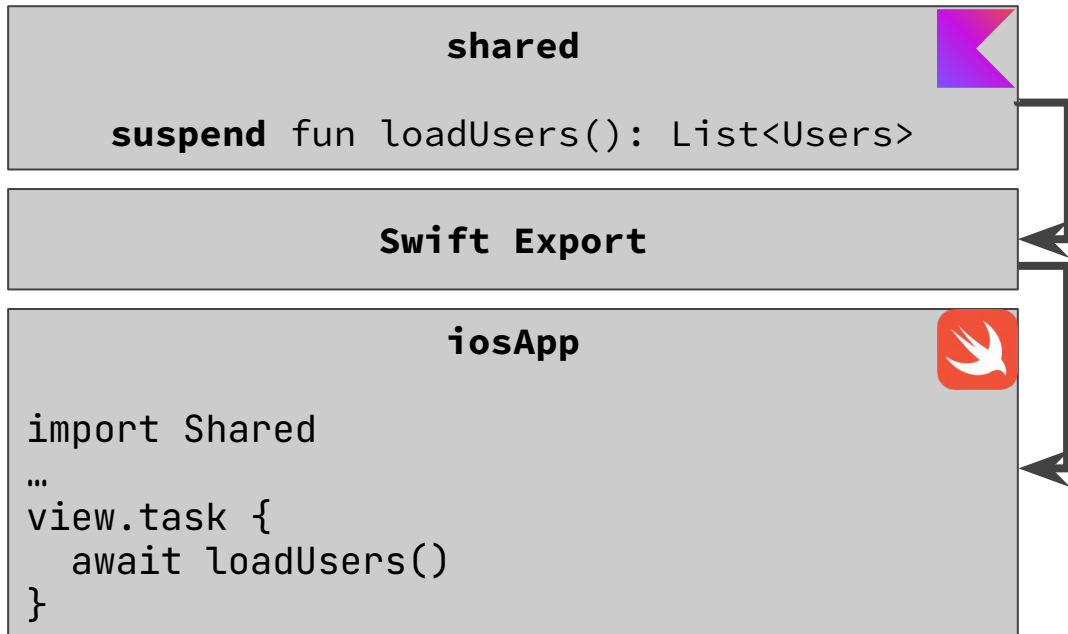
Swift Export

iosApp

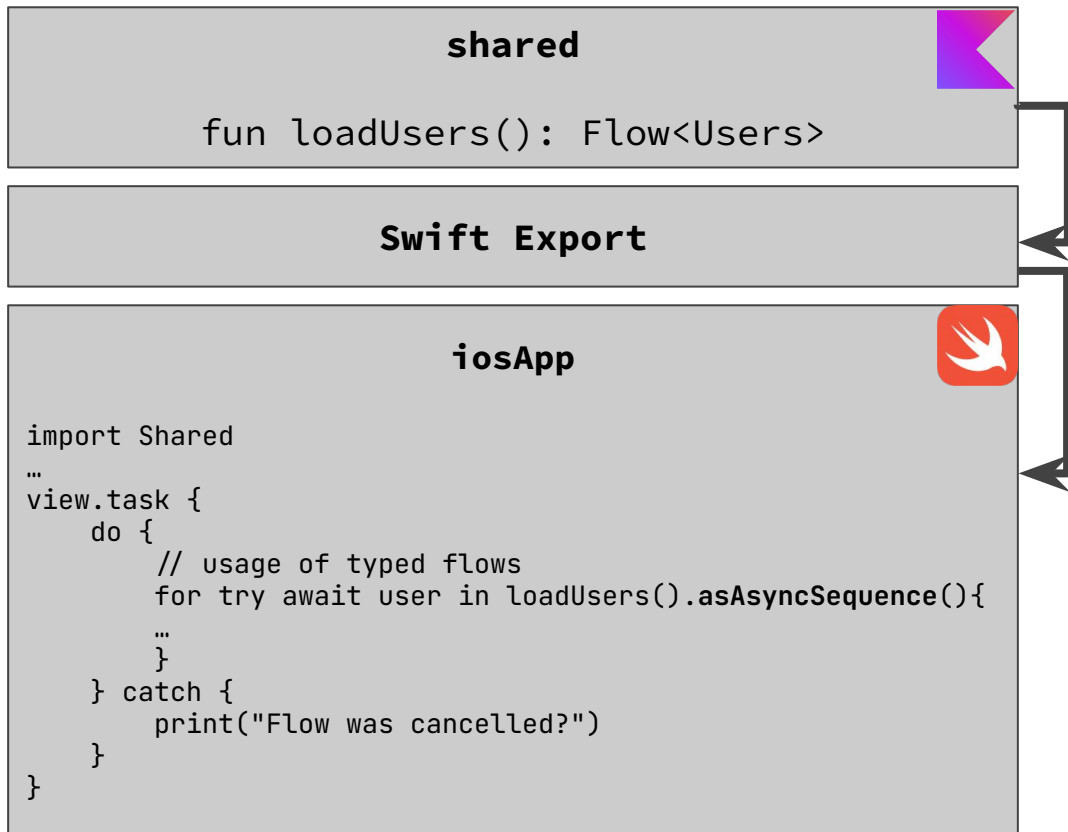
```
import Shared  
...  
callWithByte(b: Int8(10))  
callWithInt(i: Int32(100))  
callWithLong(l: 1000)  
OR  
callWithByte(b: nil)  
callWithInt(i: nil)  
callWithLong(l: nil)
```

See more in the
[Swift docs](#)

Suspend Functions



Flows



Practical 3: From Obj-C to Swift

Export with Kodee

[30 mins]

A glowing pink and purple 3D wireframe triangle is centered on a dark background. The triangle's edges are composed of bright, neon-like lines that create a strong sense of depth and perspective. The colors transition from a deep magenta at the top to a vibrant pink at the bottom, with a bright white highlight at the top vertex. The overall effect is futuristic and high-tech.

Code Quality

What we'll be looking at:

- Adding code coverage (JVM-only) with Kover.
- Modularizing your application.

Code coverage with Kover (JVM-only)

Why code coverage?

- Functionality that has not been properly tested
- Edge cases
- Dead code

BUT NOT

- Correctness of the program or test

A quick intro to Kover

- Gradle plugin from JetBrains.
- Support for Kotlin/JVM, Kotlin Multiplatform, Kotlin/Android projects.
- Support for instrumentation on JVM, not Android. Otherwise just unit tests.
- Supports multiple modules in your projects.
- HTML/XML reports – feed it into 3rd party tools like codecov (quality tool).

Setup:

1. Add the following to shared build.gradle.kts file:

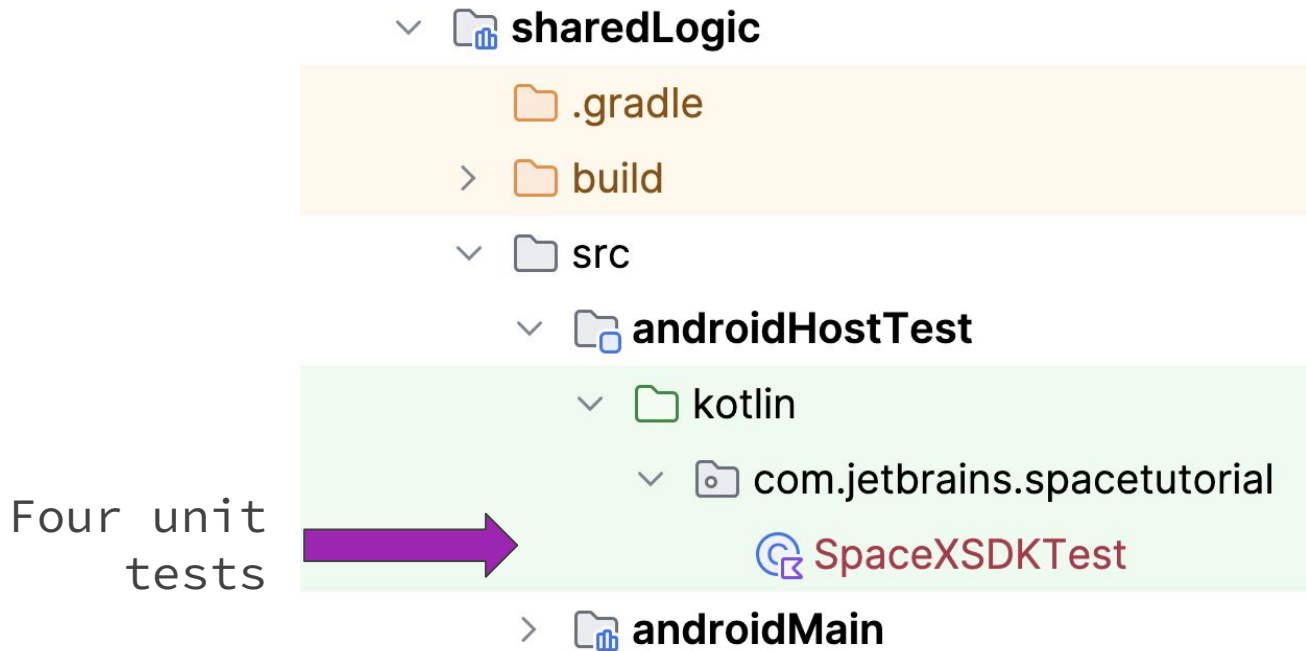
```
plugins {  
    ...  
    id("org.jetbrains.kotlinx.kover") version "0.9.8"  
}
```

2. Sync Gradle

Example: Setup unit tests dependencies

```
kotlin {...  
    sourceSets {...  
        val androidHostTest by getting {  
            dependencies {  
                implementation(libs.kotlin.test)  
                implementation(kotlin("test-junit"))  
                implementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.11.0")  
                implementation("io.mockk:mockk:1.14.9")  
            }  
        }  
    }  
}
```

Add unit tests:



Creating the report:

1. Run `./gradlew koverHtmlReport`
 - a. Where to find the report:

> Task `:sharedLogic:koverHtmlReport`

Kover: HTML report for ':sharedLogic' <file:///Users/pamelahill/StudioProjects/deep-dive-into-kmp-ultimate/newkover/sharedLogic/build/reports/kover/html/index.html>

Current scope: sharedLogic | all classes

sharedLogic: Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
all classes	66.7% (12/18)	26% (13/50)	16.7% (4/24)	27.3% (35/128)	36.9% (267/723)

Coverage Breakdown

Package 	Class, %	Method, %	Branch, %	Line, %	Instruction, %
com.jetbrains.spacetutorial	33.3% (1/3)	50% (2/4)	100% (4/4)	72.7% (8/11)	80.6% (50/62)
com.jetbrains.spacetutorial.cache	60% (3/5)	15.8% (3/19)	0% (0/6)	8.2% (6/73)	5.7% (18/318)
com.jetbrains.spacetutorial.cache.sharedLogic	66.7% (2/3)	33.3% (2/6)		40% (4/10)	50% (16/32)
com.jetbrains.spacetutorial.entity	100% (6/6)	37.5% (6/16)	0% (0/14)	63% (17/27)	66.5% (183/275)
com.jetbrains.spacetutorial.network	0% (0/1)	0% (0/5)		0% (0/7)	0% (0/36)

Current scope: sharedLogic | [all classes](#) | com.jetbrains.spacetutorial

Coverage Summary for Package: com.jetbrains.spacetutorial

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
com.jetbrains.spacetutorial	33.3% (1/3)	50% (2/4)	100% (4/4)	72.7% (8/11)	80.6% (50/62)

Class 	Class, %	Method, %	Branch, %	Line, %	Instruction, %
AndroidPlatform	0% (0/1)	0% (0/1)		0% (0/2)	0% (0/8)
Platform_androidKt	0% (0/1)	0% (0/1)		0% (0/1)	0% (0/4)
SpaceXSDK	100% (1/1)	100% (2/2)	100% (4/4)	100% (8/8)	100% (50/50)

```
1 package com.jetbrains.spacetutorial
2
3 import com.jetbrains.spacetutorial.cache.Database
4 import com.jetbrains.spacetutorial.cache.DatabaseDriverFactory
5 import com.jetbrains.spacetutorial.entity.RocketLaunch
6 import com.jetbrains.spacetutorial.network.SpaceXApi
7
8 class SpaceXSDK(databaseDriverFactory: DatabaseDriverFactory, val api: SpaceXApi) {
9     private val database = Database(databaseDriverFactory)
10
11     @Throws(Exception::class)
12     suspend fun getLaunches(forceReload: Boolean): List<RocketLaunch> {
13         val cachedLaunches = database.getAllLaunches()
14         return if (cachedLaunches.isNotEmpty() && !forceReload) {
15             cachedLaunches
16         } else {
17             api.getAllLaunches().also {
18                 database.clearAndCreateLaunches(it)
19             }
20         }
21     }
22 }
```

Demo time

Modularization

What is modularization?

- Organizing code into loosely coupled and self contained parts - **modules**.
- Each module is independent and has a clear purpose.
- By dividing the problem domain into smaller subproblems, reducing the design/maintenance complexity of the larger system.

Why modularization?

- **Reusability** of modules between apps.
- **Strict visibility controls** between modules (using internal/private modifiers) .
- **Scalability** - changes are mostly localised.
- **Possibly ownership** - dedicated owner for a module responsible for maintenance and further development.
- **Encapsulation** - isolated code is easier to read and understand.
- **Testability** - components can be easily tested in isolation.
- **Build time improvements** - some Gradle functionality can leverage modularity.
 - Incremental build, build cache, parallel build

Strategies

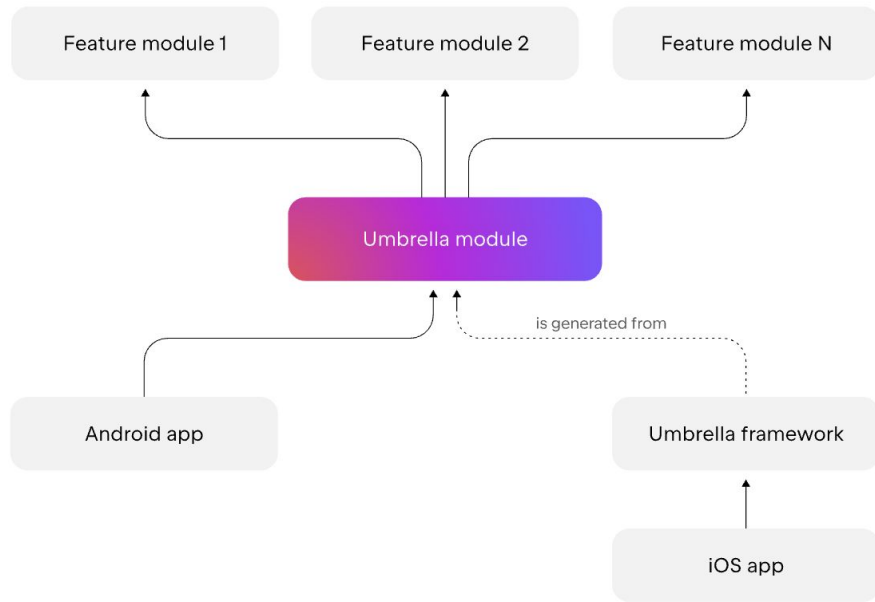
- Layer division - application's architectural layers such as presentation / domain / data.
- Feature division - different features like camera / filters / upload.

Common pitfalls

- Too fine-grained - many small modules.
 - Each module has overhead of build complexity/boilerplate.
 - Complex/distributed build configuration is difficult to keep consistent across modules.
 - Too much boilerplate makes codebase cumbersome/difficult to maintain.
- Too coarse-grained - too large modules, monolith.
 - Miss benefits of modularization.
- Too complex - size of codebase doesn't warrant it.
 - The codebase is small and won't grow too much.

Umbrella framework 🌂

- iOS apps can only depend on one KMP framework.
- For several modules, you need an extra module depending on all of the modules you're using – an **umbrella module**.
- An **umbrella framework** is a framework containing all the project's shared modules. This framework is imported into the iOS app.
- Android app can depend on Umbrella module for consistency.



Why an umbrella framework 🌂?

- When a KMP module is compiled into a framework, the resulting framework **includes all of its dependencies**.
- Whenever two or more modules use the same dependency and are exposed to iOS as separate frameworks, the Kotlin/Native compiler **duplicates the dependencies** (for example, the Kotlin stdlib might be duplicated).
- This duplication causes a number of **issues**:
 - iOS app size is unnecessarily inflated.
 - A dependency's code structure is incompatible with the duplicated dependency's code structure.

Additional resources

- [Guide to Android app modularization](#)
- [Demo of setting up an umbrella module](#) – Marton Braun
- [Do I need an umbrella framework for my KMP app?](#)
- [Swift package export setup](#)



App Quality

What we'll be looking at:

- Fixing memory leaks.
- App size optimization (for iOS).

Fixing memory leaks

Memory management in Kotlin

- Garbage collection.
- Automatic memory management where the GC reclaims memory occupied by unreferenced (ie. not strongly referenced) objects.
- This frees up memory for other objects.
- The GC runs according to policies, but you shouldn't try to guess when/if it will run.

Memory management in Objective-C and Swift

- Reference counting.
- Stores the number of references to an object, and releases the memory when it reaches zero.

Some terminology

- A **retain cycle** occurs when a program consistently references an object, and this chain of references is **never broken**.
- **Strong references** prevent an object from being deallocated.
- **Weak references** do not prevent an object from being deallocated.
- **Soft references** are weak references but will not be collected until memory constraints require it.

Brainteaser 🤖 - where and why is this code leaking memory?

```
val superview = UIView()
val subview = object : UIView() {
    val ref = superview
}
...
superview.addSubview(subview)
```

What about now? 🧠

```
val superview = UIView()
val subview = object : UIView() {
    val ref = superview
}
...
superview.addSubview(subview)
```

Retain cycle between Kotlin and Obj-C/Swift

```
val superview = UIView()
val subview = object : UIView() {
    val ref = superview
}
```

Primary source of the leak

...

```
superview.addSubview(subview)
```

Kotlin (strong) references an ObjC object and ObjC references Kotlin object. This causes a retain cycle!

Breaking the cycle with WeakReference

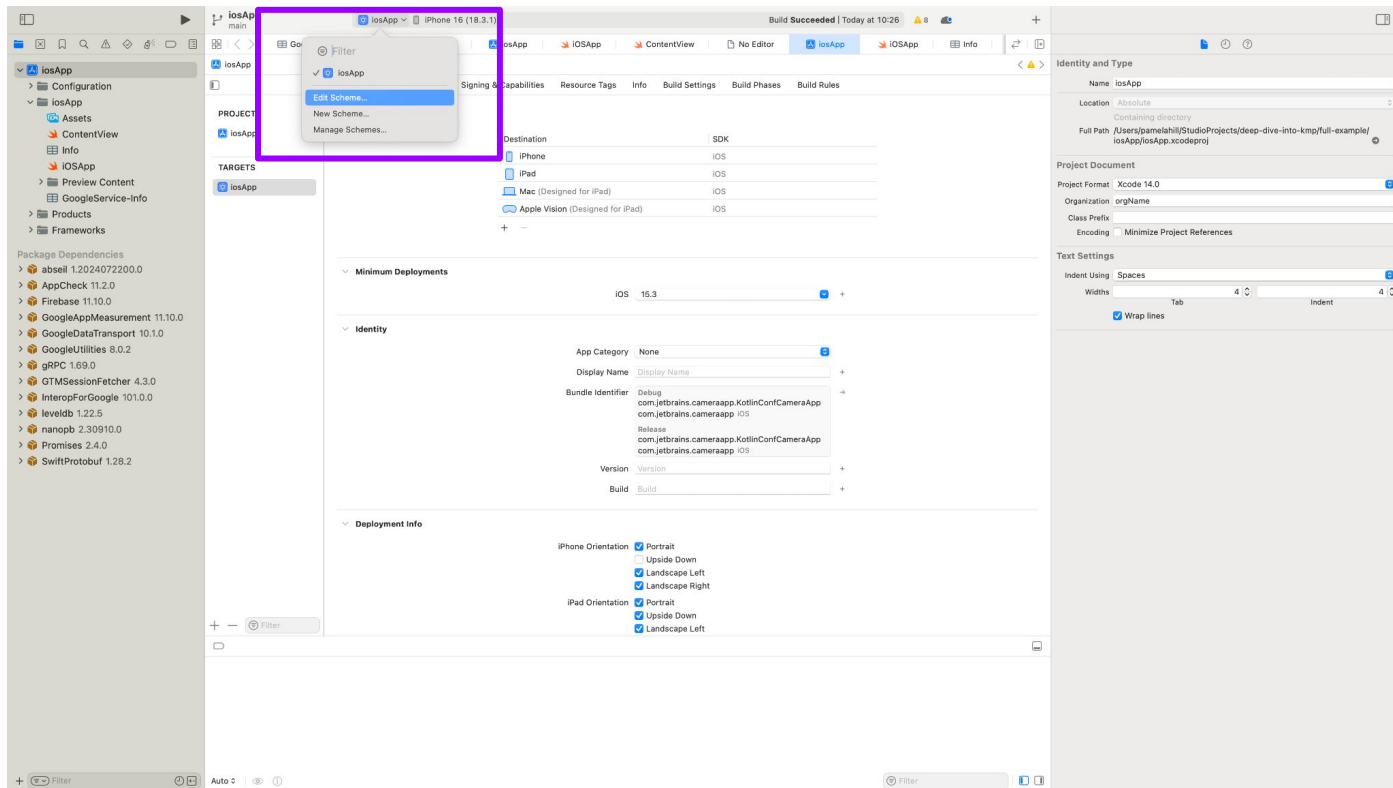
```
val superview = UIView()
val subview = object : UIView() {
    val ref = WeakReference(superview)
}
...
superview.addSubview(subview)
```

Demo time

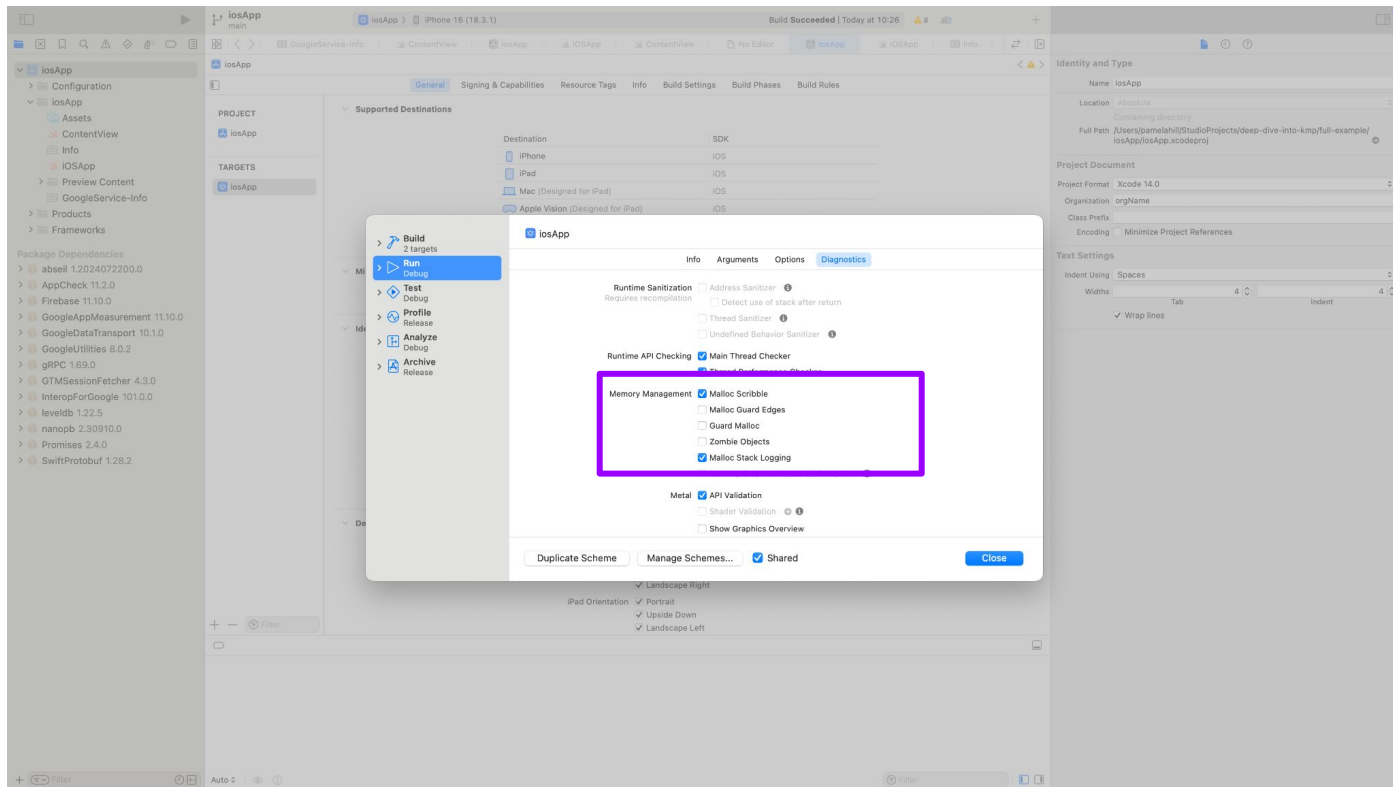
Steps to debug a memory leak in Xcode

1. Enable malloc debugging info on the scheme.
2. Run the application.
3. Open the debug navigator (memory section).
4. Create the debug memory graph.
5. Filter to only the leaked allocations.
6. Select the leak.
7. View leak backtrace.
8. Fix the memory leak and review!

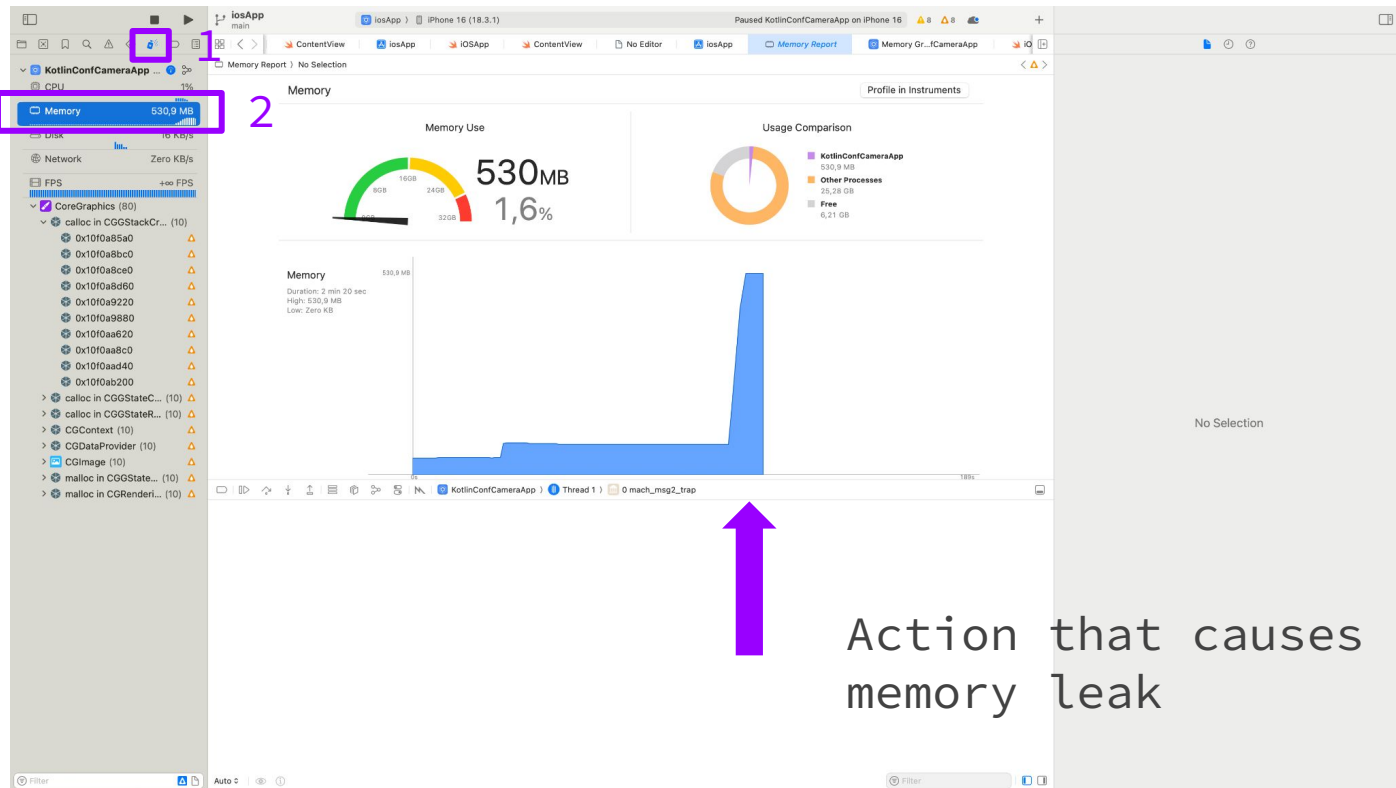
Enabling malloc debugging info



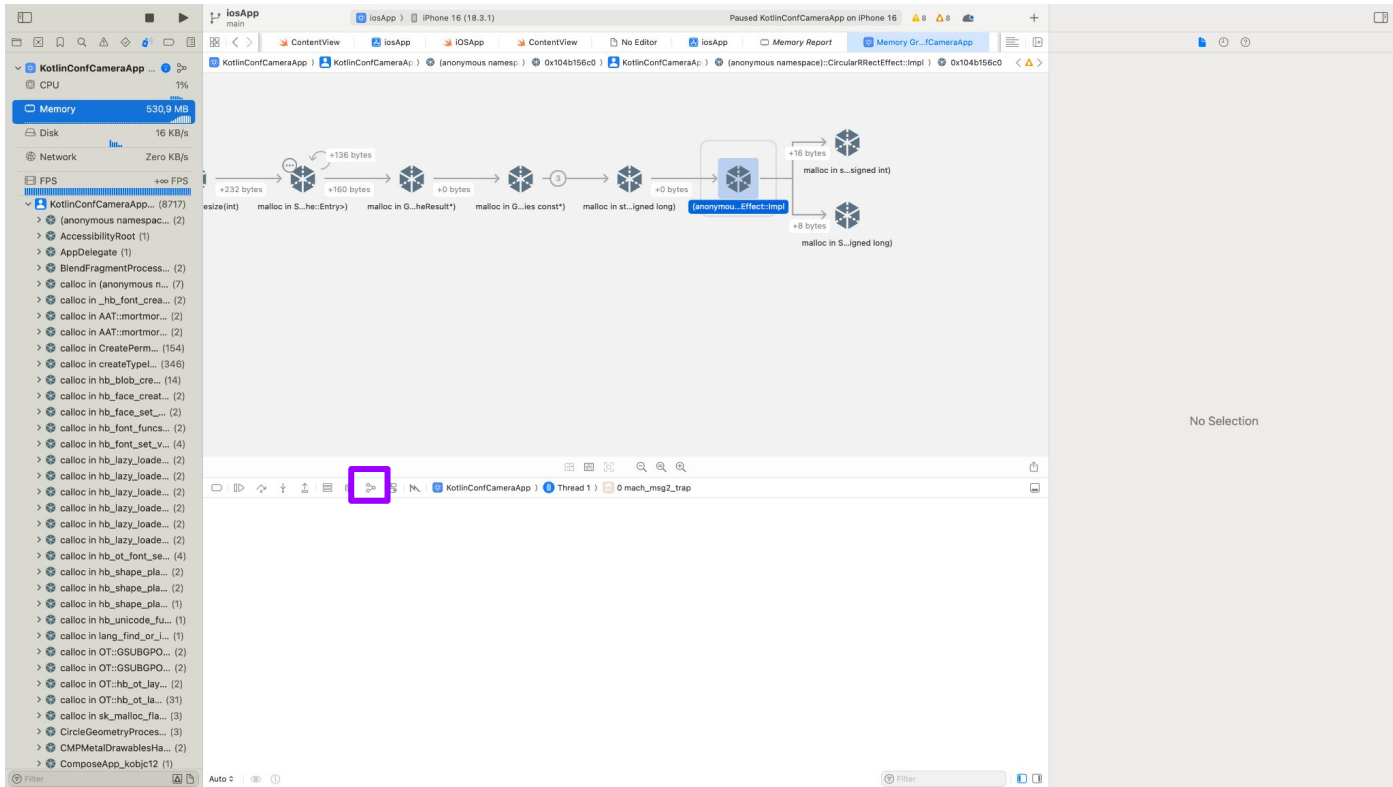
Enabling malloc debugging info



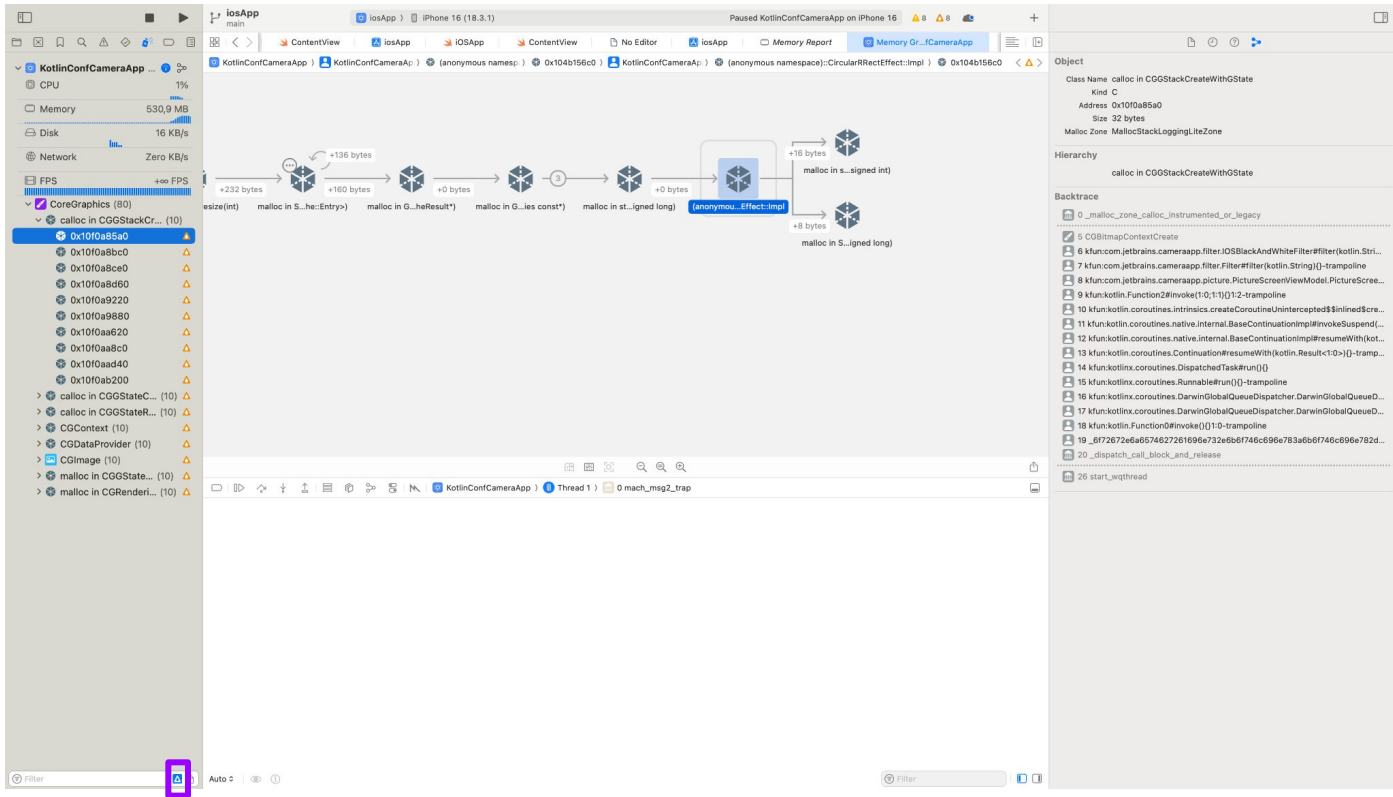
Run, then look at memory in Debug Navigator



Create Debug Memory Graph



Show only leaked allocations



Select leak

The screenshot displays the Xcode Memory Report tool for an iOS application named 'iosApp' running on an iPhone 16 (18.3.1). The interface is divided into three main sections:

- Left Sidebar:** Shows the 'Memory' tab with a list of memory allocations. The '0x1010a85a0' address is highlighted in a purple box. Below it, a list of allocations is shown, including 'calloc in CGStackCreateWith0State' (10), 'CGContext (10)', 'CGDataProvider (10)', 'CGImage (10)', 'malloc in CGStackState' (10), and 'malloc in CGRender' (10).
- Main Pane:** Displays a call graph for the 'malloc in s..._he-Entry>' function. The graph shows a sequence of calls, with the 'malloc in s..._he-Entry>' function being the most recent. The 'malloc in s..._he-Entry>' function is highlighted in a blue box.
- Right Pane:** Shows the details of the selected object, 'calloc in CGStackCreateWith0State'. The object's class name is 'CGStackCreateWith0State', its kind is 'C', its address is '0x1010a85a0', its size is '32 bytes', and its malloc zone is 'MallocStackLoggingLiteZone'. The hierarchy shows 'calloc in CGStackCreateWith0State'. The backtrace lists the stack frames, starting with 'CGStackCreateWith0State' and ending with 'start_wqthread'.

View leak backtrace

The screenshot shows the Xcode interface with the Memory tool open. On the left, the 'Memory' pane shows a list of memory allocations, with 'CGContext' (0x10b6e4540) selected. A purple arrow labeled 'Select' points from this entry to the 'Object' pane on the right. The 'Object' pane shows the details of the selected object, including its class name 'CGContext', kind 'CFType', address '0x10b6e4540', size '224 bytes', and malloc zone 'MallocStackLoggingLiteZone'. Below this, the 'Hierarchy' pane shows the object's parent, 'CGContext'. The 'Backtrace' pane shows the stack of calls that led to the leak, with the first two items highlighted in purple:

1. Leaked object type is CGContextCreate
2. In the B&W class' filter function

Fix the leak by releasing the context manually

```
@OptIn(ExperimentalForeignApi::class)
class IOSBlackAndWhiteFilter : BlackAndWhiteFilter{
    override fun filter(imagePath: String) {
        """
        val context = CGContextCreate(
            """
        if (context != null) {
            """
            CGContextRelease(context)
        } else {
            """
        }
    }
}
```


Reminder of the steps

1. Enable malloc debugging info on the scheme.
2. Run the application.
3. Open the debug navigator (memory section).
4. Create the debug memory graph.
5. Filter to only the leaked allocations.
6. Select the leak.
7. View leak backtrace.
8. Fix the memory leak and review!

Practical 4: Fixing a memory leak

[30 mins]

Additional resources

- [GC you later, Allocator - Jesse Wilson](#)
- [\(Kotlin docs\) Integration with Swift/Objective-C ARC](#)
- [Memory Management in Swift - Furkan Yildirim](#)

iOS App Size Optimization

A few tricks from us

- Make sure your API surface is minimal - use `internal` or `@HiddenFromObjC` keywords.
- Using a big library for a small function? Rather rewrite it.
- [smallBinary configuration](#) to decrease binary sizes for releases.
- No transitive exports if you can help it!

Additional resources

- [\(Apple docs\) Reducing your app's size](#)
 - How to consistently, correctly measure app size.
 - Links to articles on basic/advanced strategies.
 - Asset optimization for images/video/audio

A glowing pink and purple geometric shape, resembling a stylized triangle or a corner of a room, set against a dark background. The shape is composed of several bright, neon-like lines that form a triangular structure with a smaller triangle inside it. The colors transition from a deep purple on the left to a bright pink on the right. The text "Q&A with Kostya" is centered within this shape.

Q&A with Kostya

A glowing neon triangle in shades of pink and purple is centered on a dark, gradient background. The triangle's edges are bright and slightly blurred, giving it a three-dimensional, ethereal appearance. In the center of the triangle, the words "Discussion Time" are written in a clean, white, sans-serif font.

Discussion Time

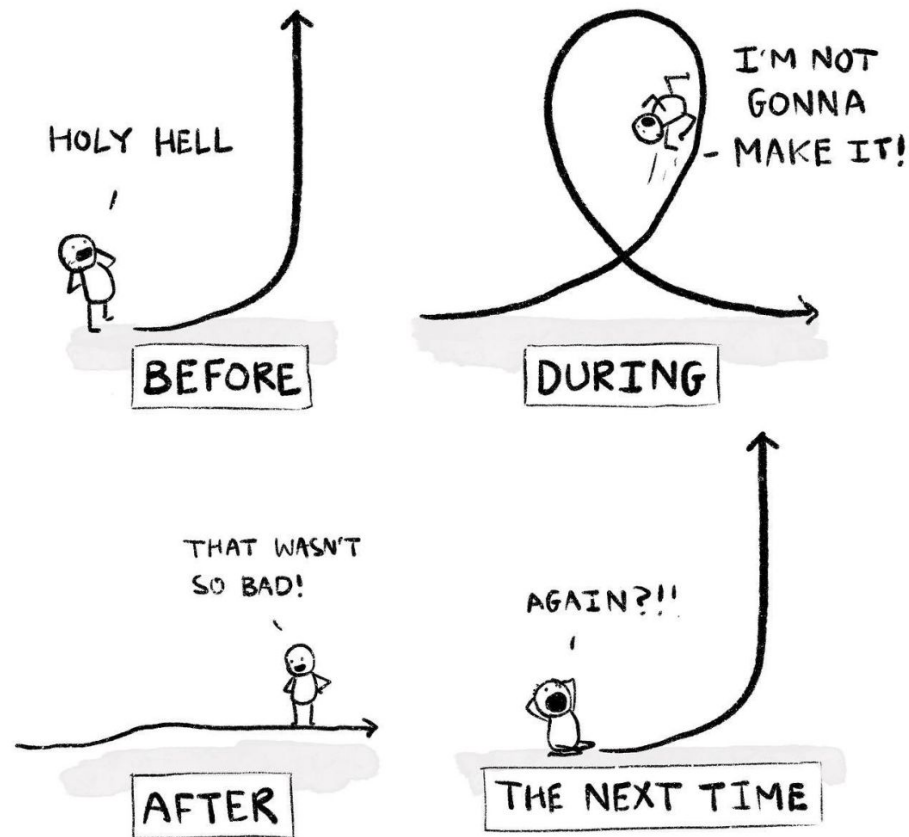
What did you like learning
about the most?

What are you taking back to
your team 🤝?

A glowing, neon-like geometric shape, possibly a stylized letter 'C' or a triangle with a missing side, is centered on a dark background. The shape is composed of several bright pink and purple lines that form a continuous, glowing outline. The word "Closing" is written in a white, sans-serif font across the middle of the shape.

Closing

HOW LEARNING CURVES FEEL...



Group picture



Let me know if you don't want to be photographed

<https://developer.apple.com/download/all/?q=xcode>

Rename to Xcode_26_5.app and put in Applications folder

```
sudo xcode-select -switch /Applications/Xcode_26_5.app
```

Download the cmd line tools too

Install the pkg

In Android Studio Settings > Tools > Xcode
Select 26.5 (if it's not there yet, restart AS)

Restart + test

